# DSA4213 Assignment 1: Do Better Next-Word Predictors Give Better Sentiment Features, and at What Cost?

**Lee Kai Rong, Student ID: A0287723U**
Code Repo: https://github.com/kr7001/dsa4213-assignment

## Abstract

Do language models that predict the next word better also give better sentiment features, and what does the better prediction cost? On Amazon beauty reviews, a Kneser-Ney 5-gram and an RNN, LSTM and Transformer of equal core size (0.37M parameters) reach test perplexity 60.9, 50.1, 40.8 and 41.2; the neural models train for 25 to 30 minutes, the 5-gram for 14 seconds. Fixing the LSTM's input word vectors instead of learning them raises its perplexity by 11% with my own skip-gram vectors and by 29% with GloVe, despite GloVe's 1,100 times larger corpus. I then freeze 19 networks and train a linear classifier on the 100-number vector just before each network's output layer, taken after a review's last word or averaged over all words. Across the 13 distinct models (78 pairs), the better next-word predictor is also the better classifier in 66 pairs with last-word vectors (Kendall's $\tau = 0.69$) and in 72 with averaged ones ($\tau = 0.85$). The RNN is the main exception: its last-word vectors score 4 points below its averaged ones, consistent with its short memory in Part I, where its lead over the 5-gram stops growing after about five tokens. The best features (93.4% accuracy) match, but do not beat, tf-idf counts of words and word pairs (93.6%), which need no language model.

## 1 Question and Setup

One question runs through all three parts: *does a language model that predicts the next word better also give better features for sentiment classification, and what does the better prediction cost?* The answer is open: GPT-1's zero-shot sentiment accuracy rose steadily with pre-training (Radford et al., 2018, Sec. 5), but models with equal pre-training loss can differ downstream (Liu et al., 2023). Part I compares model designs and their costs, Part II input word vectors; together they train 19 neural language models whose test cross-entropy $J$ (Eq. 1; lower is better) ranges from 3.57 nats (best) to 4.03 (worst), a spread wide enough to test the question. Part III freezes all 19 and tests, with one linear classifier, whether lower $J$ gives higher sentiment accuracy. Every setting was fixed before the results it affects, from a measurement on this data or published evidence, with three exceptions (Appendix A; each decision's rejected alternatives and evidence: Tables A1 and A2). Results paragraphs give observations first, then **possible explanations**. Code, seeds and AI use: Appendix G.

**Corpus.** I chose the All_Beauty reviews of Amazon Reviews 2023 (Hou et al., 2024) over six other corpora (comparison: Table B1) for three reasons: (1) star ratings give in-domain sentiment labels for Part III; (2) all reviews of a product can be kept on one side of every split, so no test product reaches language-model training; (3) reviews are unlikely to be in GloVe's Wikipedia and Gigaword text, so Part II tests GloVe on text it has not seen. Text is lowercased, numbers become <num> (so prices and sizes need no vocabulary entries), and the course tokenizer (Zhou, 2026a) keeps words, contractions and six punctuation marks.

**Filters.** From 701,528 raw reviews, four filters remove, each with a threshold measured on this data (counts: Table B2): (i) empty reviews and reviews over 512 tokens, so one Transformer context holds any review; short reviews stay, as a 3-word minimum would remove 8.7% of positive but only 3.4% of negative reviews and skew Part III's labels (Figure B1a); (ii) non-English reviews, as the brief asks for English: the usual detector fastText is unreliable on very short reviews (in a sample of 25 one- or two-word reviews it judged non-English with under 50% confidence, only 11 were), so reviews of up to six words are judged by my own word score, which measures how much more common their words are in non-English reviews (Tables B3 and B4); (iii) a person's repeated reviews and identical reviews of 14+ words by different people, as shorter matches such as "love it" are everyday phrases (Figure B1b); (iv) reviews beyond 90 per product, so popular products cannot dominate (the top 1% held 28% of reviews); as reviews of one product are alike, 90 keep 95% of what they tell about its rating (Figure B1c; Kish, 1965). 587,133 reviews (21.2M

tokens) remain (checks: Table B5).

**Splits and vocabulary.** To prevent leakage, I split by product, not by review: each product gets a random key (seed 4213) that sends all its reviews to one of five piles: language-model training (50% of products; 10.96M targets), validation (5%; 1.07M), test (5%; 1.12M), Part III test (10%) and Part III training (30%). So no evaluation or Part III product is seen in language-model training, and 5% is enough for validation and test: it measures $J$ to within about 0.003 nats (one standard error), far below the gaps compared here (sizes: Appendix B). The vocabulary is every token seen 5+ times in language-model training, plus <pad> (batching), <unk> (unseen tokens), <bos> and <eos> (review start and end), $|V| = 17{,}068$. A cut-off of 5 leaves only 0.60% of test targets as <unk>, so a lower one would add thousands of words too rare to learn while removing few unknowns (Figure B1d); 5 is also gensim's default, so Part II's skip-gram sees every word at least five times.

**Scoring.** All models face the same prediction problem (Zhou, 2026b, slide 29): each review is scored on its own from $x_1 =$ <bos>, so no model uses another person's review as context; review $r$ has $T_r$ targets (its tokens and <eos>), $N = \sum_r T_r$, and

$$J_{\text{test}} = \frac{1}{N} \sum_{r} \sum_{t=1}^{T_r} -\log P_\theta\big(x_{t+1}^{(r)} \,\big|\, x_{1:t}^{(r)}\big), \qquad (1)$$

with $\text{PPL} = \exp(J_{\text{test}})$. $J$ is in nats (natural-log units); 0.1 nats less is about 10% lower perplexity. Padding is masked, and the code asserts the same $N$ for every model (1,115,210 test targets).

## 2 Part I: Four Language Models

**Models.** The $n$-gram baseline is my own interpolated modified Kneser-Ney 5-gram, the best smoothing in Chen and Goodman's (1998) comparison, with the course's add-$k$ 5-gram (Zhou, 2026a) as a second baseline; validation picks their settings. Each neural design has two sizes: S (main) and M, with about 4 times the core parameters, to check whether the ranking of designs survives a change of size. The RNN (Elman, 1990) and LSTM (Hochreiter and Schmidhuber, 1997) have one recurrent layer (S: 512 and 248 units) and carry $h^{(t)}$ (and $c^{(t)}$) through the whole review; the Transformer (S: three causal pre-LN layers, $d_{\text{model}} = 100$, 4 heads) sees up to 513 tokens, which holds every review, so each model uses its natural context. To compare designs, not sizes, all three have the same word tables (input table $E$, $|V| \times 100$, and an untied output layer; 3.4M parameters) and cores, everything else, within 2.3% of each other in size (Kaplan et al., 2020; Collins et al., 2017); the unit counts are set to meet that budget (Table C1), since equal hidden units would give unequal sizes. $d_{\text{model}} = 100$ matches Part II's GloVe vectors; pre-LN (Xiong et al., 2020) with sinusoidal positions needs no long warm-up or position parameters (Table A1).

**Training.** The three networks share training settings, each chosen from evidence before any result (source for each: Table C2): AdamW (weight decay 0.1); batches of 8,192 token positions, beyond which five of six models stopped training faster in a pilot run (Figure C1); a 4% warm-up, then cosine decay over 6 epochs, the longest budget that let all tuning tries finish overnight; clipping at 1.0; and validation $J$ 4 times per epoch, keeping the best checkpoint. Dropout is 0: with 0.35 parameters per training target (AWD-LSTM: 16 to 26; Merity et al., 2018), overfitting was unlikely, and a rule written before training would have added dropout to all three if any model overfitted before the last quarter of its run (rules: Appendix C). Only the peak learning rate is tuned, by one search for all models with every try a full run, since the winner can depend on the tuning budget (Dodge et al., 2019) (every try: Table C3).

**Prediction quality.** On test cross-entropy $J$, every S model beats the 5-gram: the LSTM and Transformer by about 0.4 nats (perplexity about 41 against 61), the RNN by 0.20 (Table 1). The LSTM and Transformer are only 0.010 nats apart, about 2.6 standard deviations of seed noise (estimated from only three LSTM-S seeds; Section 3), so with one run each I do not rank them firmly; at M the Transformer leads by a clearer 0.032, and the RNN gets worse with size. $J$ by position for the S models (Figure D1, Table D1) shows where the neural models gain: on the first 4 targets, where the 5-gram sees the whole history, they lead it by only 0.04 to 0.10 nats, so little of their edge comes from better word statistics; from target 5, where the 5-gram's window starts to drop words, the lead jumps to 0.22 (RNN) and 0.38 (LSTM, Transformer), so it comes from longer context; and only the LSTM's and Transformer's lead keeps growing, so only they use distant context. At S the Transformer is better on targets 1 to 8 and the LSTM from target 33; the LSTM's whole 0.010 lead comes from targets be-

| | Test | | | Size and arithmetic | | | K targets/s | | Training | |
|---|---|---|---|---|---|---|---|---|---|---|
| Model | $J$ | PPL | Peak LR (tries) | Core | MFLOP | MB | GPU | CPU | min | tries (h) |
| Kneser-Ney 5-gram | 4.110 | 60.9 | – | – | – | 425 | – | 3,425 | 0.24 | – |
| Add-$k$ 5-gram | 4.280 | 72.2 | $k$=$10^{-5}$ | – | – | 425 | – | – | 0.80 | – |
| **Size S (main)** | | | | | | | | | | |
| RNN-S | 3.914 | 50.1 | $3\times10^{-3}$ (4) | 365,668 | 4.14 | 15.2 | 144 | 45.2 | 25.1 | 1.71 |
| LSTM-S | 3.708 | 40.8 | $10^{-2}$ (5) | 372,100 | 4.15 | 15.2 | 148 | 42.9 | 24.7 | 2.56 |
| Transformer-S | 3.718 | 41.2 | $3\times10^{-3}$ (4) | 374,200 | 4.21 | 15.2 | 139 | 45.5 | 29.7 | 2.37 |
| **Size M (size check)** | | | | | | | | | | |
| RNN-M | 4.001 | 54.7 | $10^{-3}$ (3) | 1,480,740 | 6.37 | 19.6 | 123 | | 32.2 | 1.97 |
| LSTM-M | 3.602 | 36.7 | $10^{-2}$ (5) | 1,489,500 | 6.38 | 19.7 | 123 | | 29.0 | 3.01 |
| Transformer-M | 3.570 | 35.5 | $3\times10^{-3}$ (4) | 1,578,980 | 6.70 | 20.1 | 120 | | 36.4 | 3.03 |
| **Size S, 10 epochs (budget check)** | | | | | | | | | | |
| RNN-S | 3.855 | 47.2 | $3\times10^{-3}$ | | | | | | 42.2 | |
| LSTM-S | 3.684 | 39.8 | $10^{-2}$ | | | | | | 46.2 | |
| Transformer-S | 3.700 | 40.4 | $3\times10^{-3}$ | | | | | | 45.2 | |

Table 1: Part I on the common test setup ($N = 1{,}115{,}210$ targets). *Peak LR (tries):* chosen learning rate and number of full-run tries. *Core:* parameters outside the word tables (3,430,668 more each). *MFLOP:* per prediction at the mean test context (47 tokens). *MB:* checkpoint, or the 5-gram's 19.7M $n$-grams. *K targets/s:* scoring the test pile, median of 5 timed passes after a warm-up (GPU: M4 via MPS; CPU: 4 threads). *Training:* minutes of the kept run without checks (5-gram: counting); hours of all tries.

yond 64 (22% of the test set), so the better design depends on review length. **Possible explanations.** The RNN's flat lead fits vanishing gradients (Bengio et al., 1994), which clipping cannot fix. The small Transformer may trail on long reviews because few positions lie that far in (8% of targets are beyond 128) and its 100-number states leave little room; the deficit disappears at size M, as a capacity limit would predict (untested). RNN-M is worse than RNN-S although 4 times larger, and its gradients were clipped in 30% of steps (RNN-S 12%), which suggests it is harder to optimise rather than short of capacity.

**Cost and trade-offs.** Per prediction, the three S designs cost almost the same (Table 1): arithmetic within 2%, most of it in the shared output layer, and similar scoring speed. Their real difference is training: Transformer-S trains for 29.7 minutes against 24.7 (LSTM-S) and 25.1 (RNN-S), and tuning took 1.7 to 2.6 hours per model. The 5-gram is the opposite trade-off: it counts in 14 seconds and scores 75 to 80 times faster on the CPU, but needs a 425 MB table and predicts 0.2 to 0.4 nats worse. Size M costs 1.5 to 1.6 times the arithmetic and 17 to 28% more training time, and pays off only for the LSTM and Transformer.

**Learning behaviour.** For every model, validation $J$ still falls at the end of 6 epochs, above the training sample (Figure 1a), and every best checkpoint is one of the last two of 25 checks: no model overfits, so the dropout rule never fired. Were 6 epochs enough? The last epoch still gained 0.013 to 0.025 nats, more than the LSTM-Transformer gap, so a second rule written in advance gave all three 10-epoch runs (Figure D2): the ranking held (LSTM ahead by 0.016), and only the Transformer stopped early, at 9.7 epochs, on its first sign of overfitting (validation $J$ stalled while training $J$ fell), too late to trigger the dropout rule. **Possible explanations.** The Transformer fits its training text most closely (after 6 epochs its training sample sits 0.190 nats below validation, against 0.140 and 0.105), so without dropout it overfits first.

**Generated text.** I sample with temperature, $q_{t,T}(w) \propto p_t(w)^{1/T}$ (Zhou, 2026b, slide 21), one $T$ for all models so the comparison is matched. Rather than adopt the course's $T = 0.8$, I measured it: on 300 validation openings, $T = 1.01$ makes each Part I model's own text about as surprising to it as real text (Figure 1c), the calibration check of Holtzman et al. (2020, Sec. 4.2); at $T = 0.8$ the text would be 1.1 to 1.5 nats too predictable and repeat 4-grams up to 3.9% of the time. Each sample is 22 new tokens, a median review's length; for six prompts fixed before generation, every model writes 9 samples and I show the middle one by its own perplexity, so the shown sample is typical, not hand-picked. Over 300 test openings, neural text repeats 4-grams slightly more than real text and varies slightly less, while the 5-gram varies most and copies the most training 5-grams (all measures: Table D2). In the samples (all six prompts: Table D3), the 5-gram is fluent over a few words but drifts ("disappointed . . . a fuller,

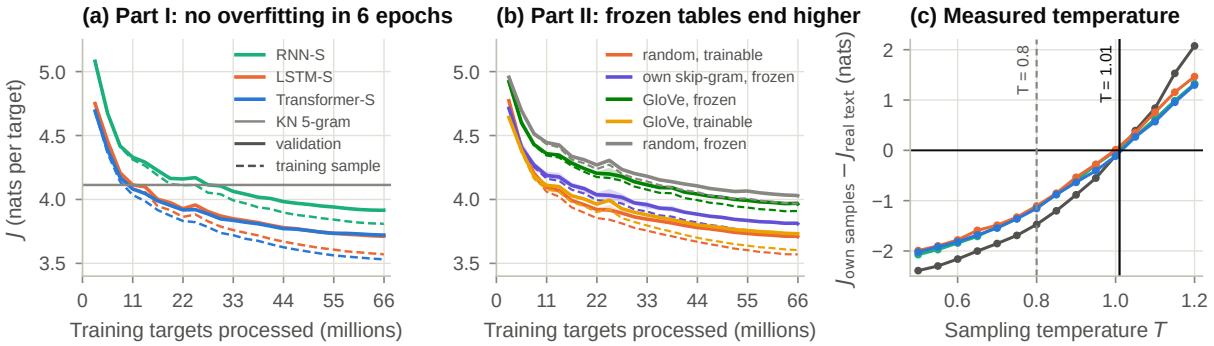



Figure 1: (a) Part I at size S: validation $J$ (solid) and a fixed 1.07M-target training sample's $J$ (dashed); ticks mark epochs. (b) Part II tables (LSTM-S): seed mean with lowest-to-highest band. (c) Each Part I model's $J$ on its own 22-token continuations of 300 validation openings minus its $J$ on the real ones (colours as in a); the models cross zero between $T = 0.998$ and 1.015.

healthier"), the LSTM keeps prompt 3's complaint going ("felt greasy . . . didn't even care for"), the Transformer keeps prompt 6's daughter as the subject ("she loves this product"), and every model breaks grammar within 22 tokens ("vibrant black black hair"). **Possible explanations.** The 5-gram sees only four previous tokens, so its text is locally plausible but globally loose, and it generates from counts, so it copies most. Low copying by the neural models is not creativity, since broken word combinations are new too.

**Part I in brief.** At equal size the LSTM and Transformer predict best (tied at S, the Transformer ahead at M), the RNN uses little distant context, and the 5-gram is cheapest but weakest. Part II uses the LSTM-S (lowest validation $J$, a rule fixed in advance); Part III asks whether these differences carry over to sentiment.

## 3 Part II: Embedding Variants

**Design.** Part II varies only the LSTM-S's input table, at $d = 100$: GloVe's 300 dimensions improve its analogies but add little as model input (CoNLL-03 F1 90.34 against 90.72; Carlson et al., 2025) and would triple the table (reasons: Table A2). Three settings: (1) a random table trained with the model (Part I's LSTM-S); (2) my skip-gram vectors (Lecture 2 settings; Mikolov et al., 2013), trained on the training pile, then frozen; (3) GloVe 2024 (Wikipedia and Gigaword, 11.9B tokens; newer words than GloVe 2014), frozen. GloVe differs from the random table in two ways, extra data and freezing, so two extra runs separate them: GloVe left trainable (extra data, no freezing) and a frozen random table at GloVe's scale (freezing, no information). My table is scaled to GloVe's root mean square, as with Adam a fixed input's scale acts like a learning rate for the first layer. The three settings run with three seeds each (the extra runs with one) from the same starting weights and batches at the Part I learning rate (0.01); fingerprints confirm frozen tables never change. GloVe covers 97.0% of the vocabulary once contractions and tokenizer splits are built from their pieces; the rest get random vectors with GloVe's variance (Kim, 2014) (coverage: Appendix E). My vectors took 9.7 minutes on one CPU thread, against about 24 per LSTM run.

| | Test $J$ | | Valid $J$, epoch | | |
|---|---|---|---|---|---|
| Input table | Mean | SD | 1 | 6 | V−T |
| Random, trainable | 3.705 | 0.003 | 4.095 | 3.709 | 0.142 |
| Own, frozen | 3.805 | 0.009 | 4.188 | 3.811 | 0.082 |
| GloVe, frozen | 3.962 | 0.010 | 4.358 | 3.965 | 0.060 |
| GloVe, trainable | 3.729 | – | 4.111 | 3.732 | 0.128 |
| Random, frozen | 4.027 | – | 4.445 | 4.029 | 0.063 |

Table 2: Part II (LSTM-S). *Test J:* mean and SD over three seeds (one in the last two rows). *Valid J:* after epochs 1 and 6. *V−T:* validation minus training-sample $J$ at the best checkpoint (higher: more overfitting).

**Results.** Freezing hurts on every seed (Appendix E), on average by 0.100 nats with my vectors and 0.257 with GloVe (Table 2): frozen in-domain vectors beat GloVe despite its 1,100 times larger corpus. The extra runs (seed 4213) show why. Freezing a table with no word information at GloVe's scale costs 0.319 nats (freezing plus a larger scale; Appendix E); at that same scale, my vectors win back 0.215 of it but GloVe only 0.053, so GloVe's knowledge helps little when frozen. Trainable GloVe starts ahead, is level after one epoch and ends 0.021 nats behind (Table 2), so its extra data buys only a head start. *Convergence:* frozen tables end higher rather than learn more slowly (Figure 1b); my vectors even lead at the first check on every seed. *Stability:* frozen settings vary more across seeds but overfit less (Table 2), with similar gradient norms, and the order holds

at each setting's best learning rate (Appendix E). *Generated text:* frozen tables copy fewer training 5-grams and vary more (Table D2), with breakdowns such as "i don't not stop my hair" (GloVe, frozen; Table D3). **Possible explanations.** Trainability matters most: 10.7M review tokens suffice to learn the table. GloVe knows more about words in general (WordSim353 0.52 against 0.39; analogies 57.5% against 22.1%; Table E1), yet frozen it helps less than my vectors, plausibly (untested) because of word sense: my neighbours of "cheap" include *cheaply, flimsy, crappy*, GloVe's *inexpensive, cheaper, affordable*, and reviews need the complaint sense. The shared $T$ acts hotter for frozen tables (own-text minus real-text $J$: +0.19 to +0.29), which may explain their less coherent samples.

**Part II in brief.** Learning the table beats freezing it, and frozen in-domain vectors beat a far larger general corpus; Part III ranks these models too.

## 4 Part III: Downstream Representations

**Setup.** Reviews with 1 to 2 stars are negative and 4 to 5 positive; 3-star reviews are left out as ambiguous, and classes are balanced so accuracy is meaningful (74,344 training and 24,362 test reviews, from products no language model saw). Each review passes through a frozen model after `<bos>`, and I take the 100 numbers that feed its output layer, the same size in every model, so no model wins by a longer vector (top hidden states give close results: Table F4). Two readouts make one vector per review: the state after its last token (word or punctuation; "last word" below), which the model uses to predict what comes next, and the average of the states after every token (never `<bos>` or padding). All 19 models enter, in groups that change one factor (design, size, training length, word table, seed). References show what the features add: untrained copies of the S designs (does training matter? Wieting and Kiela, 2019), averaged word tables (do states add to word vectors?), tf-idf counts of words and word pairs (a strong classic baseline; Wang and Manning, 2012) and review length. Every representation gets the same L2 logistic regression (Pedregosa et al., 2011) on standardised features, a linear probe, since a stronger classifier could learn the task itself and hide differences (Hewitt and Liang, 2019). Its regularisation strength $C$ comes from 5-fold cross-validation on training reviews, folds split by product as the test set is, and the test set is used once (details: Appendix F). I report accuracy with 95% bootstrap intervals ($\pm$0.3 to 0.5 points), test 51 comparisons fixed in advance with exact McNemar tests (Dietterich, 1998), Holm-corrected (Holm, 1979), and measure ranking agreement over the 13 seed-4213 models with Kendall's $\tau$: the share of the 78 model pairs in the same order by test $J$ and accuracy, minus the share in the opposite order (1: same order).

**Results.** (1) Better next-word prediction goes with better features (Figure 2): the rankings agree on 66 of 78 pairs at the last word ($\tau = 0.69$, nominal $p = 0.0005$) and 72 of 78 averaged ($\tau = 0.85$, $p < 10^{-5}$). This holds within single factors, design (3 of 3 pairs) and word table (9 and 10 of 10), and without reviews under 3 or 5 words, so neither one group nor very short reviews drive it. (2) Language-model training makes the features: untrained copies reach only 61 to 73%, and training adds 24 to 29 points at the last word. (3) The readout matters most for the RNN: its last-word state loses 4.4 (S) and 4.1 (M) points against its average, while the LSTM's and Transformer's readouts differ by at most 0.74. On reviews over 20 words RNN-S falls to 79.9% at the last word against 87.0% averaged (Figure F1). Five of the 12 out-of-order pairs involve an RNN; the other seven differ by at most 0.6 points, within seed noise (seed SD 0.31 points; Table F1). (4) Size, training length and word table mostly move accuracy the same way as $J$; for example, frozen GloVe loses 2.9 points to the trainable table at the last word and my frozen vectors only 0.3, the same order as in Part II (36 of the 51 fixed comparisons significant after correction: Table F5). (5) The best features, LSTM-M and Transformer-M at the last word (93.4% and 93.3%; all accuracies: Table F1), match but do not beat tf-idf counts (93.6%; exploratory McNemar $p = 0.29$ and 0.07), which need no language model but 153,041 features instead of 100. **Possible explanations.** To predict a review's next words a model must track the writer's opinion, so a better predictor may store more sentiment in its state; this is a consistent correlation across groups that change one factor, not a causal test. The RNN's last state may keep little of a long review's start (Part I); averaging keeps every position.

**Errors.** To see where the best predictor (Transformer-M, last word) fails, I compare groups defined before reading any error (Table 3): reviews with a contrast word such as "but" (25% of the test set) fall to 87.6% from 95.2%, 4-star reviews to

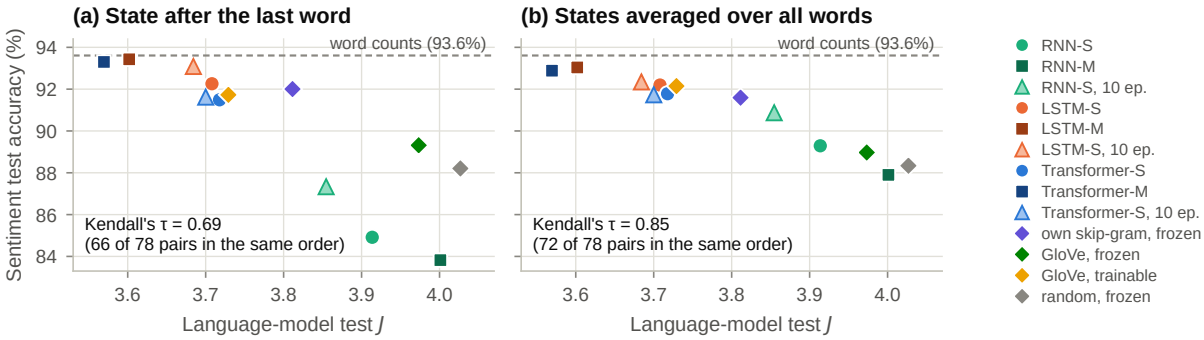



Figure 2: Sentiment accuracy against test $J$ for the 13 seed-4213 models, read at the last word (a) or averaged (b). Diamonds: Part II's word tables, all on LSTM-S. Untrained copies (61 to 73%) and length alone (52%) lie below the plots (every accuracy: Table F1).

| Test reviews | $n$ | Main | Counts |
|---|---|---|---|
| All test reviews | 24,362 | 93.3 | 93.6 |
| 4 stars | 2,013 | 78.6 | 82.8 |
| Contrast word (but, though, ...) | 6,033 | 87.6 | 88.7 |
| No contrast word | 18,329 | 95.2 | 95.2 |
| Negation word (not, n't, ...) | 12,262 | 92.9 | 93.1 |

Table 3: Accuracy (%) of Transformer-M at the last word (Main) and of word counts in groups of test reviews defined before reading any error.

78.6%, while negation costs little; word counts fail in the same groups, so these reviews are hard for both. Yet the two err on different reviews: each is right on about 850 that the other gets wrong, and at least one is right on 97.0%, so they are complementary. To avoid hand-picked examples, I categorised 40 random mistakes (counts: Table F3): the largest group (17) weigh real good and bad points; others state a complaint as plain fact ("leaves a greasy residue."), praise through "not" ("a light scent. not overpowering"), or dismiss their one good point ("the packaging was nice but that's the only plus").

**Possible explanations.** In my reading, most remaining errors would puzzle a human too: mixed reviews rated by how the writer weighs the points, and 4-star reviews that list complaints.

## 5 Synthesis, Limitations and Lessons

**The answer.** Mostly yes: across design, size, training length and word table, the better next-word predictor gives better sentiment features in 85 to 92% of model pairs, and in every pair at least 0.05 nats apart once features are averaged (exploratory; Table F2). The main exception links Parts I and III: the RNN, whose lead over the 5-gram stops growing after about five tokens, scores 7 points lower at its last word than averaged on reviews over 20 words. Part II's ranking also carries over: trainability beats 1,100 times more general text. The better prediction costs 17 to 22% more training time and 1.5 to 1.6 times the arithmetic (LSTM-M, Transformer-M), and buys features that match word counts with 100 numbers learnt without labels.

**Limitations.** Each Part I model has one seed, so close gaps (LSTM-S and Transformer-S) stay unranked; all models are small and see one domain. The 13 ranked models share data and designs, so the $J$-accuracy link is a consistent correlation, not a causal test. A linear probe shows what frozen features hold, not what fine-tuning would reach. Preprocessing limits are measured in Table B6.

**Future work.** Next I would add seeds and sizes, shuffle distant words to test the RNN's context, and combine counts with model features, which err on different reviews (at least one is right on 97.0%).

**Lessons learnt.** (1) *Measure, don't borrow:* the course's $T = 0.8$ would have made every model's text 1.1 to 1.5 nats too predictable, so I measured $T$ on this data, as I did for the filters and the vocabulary cut-off. (2) *Fix the rules before the results:* the product split, equal core sizes, one tuning search and rules written in advance (Appendix C) kept the comparisons fair; a rule, not my judgement, chose 10 epochs. (3) *Look inside averages:* one $J$ hid the LSTM's and Transformer's different strengths, one accuracy hid that word counts and the best model fail on different reviews, and the last-word readout made the RNN look weaker than its averaged states are. (4) *Relevant data beats more data:* when frozen, my vectors from 10.7M review tokens beat GloVe's from 1,100 times more text, plausibly because they learn the reviewers' sense of words such as "cheap". (5) *Keep a simple baseline:* word counts matched the best features and kept my claims about pre-training honest. (6) *Automate the evidence:* scripts write every table and check every number in the text against the results, so revisions stay consistent.

## References


Roberto J. Bayardo, Yiming Ma, and Ramakrishnan Srikant. 2007. Scaling up all pairs similarity search. In *Proceedings of the 16th International Conference on World Wide Web*, pages 131–140.

Yoshua Bengio. 2012. Practical recommendations for gradient-based training of deep architectures. In *Neural Networks: Tricks of the Trade*, 2nd edition, pages 437–478. Springer.

Yoshua Bengio, Patrice Simard, and Paolo Frasconi. 1994. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166.

Johan Bjorck, Alon Benhaim, Vishrav Chaudhary, Furu Wei, and Xia Song. 2025. Scaling optimal LR across token horizons. In *International Conference on Learning Representations*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33*, pages 1877–1901.

Riley Carlson, John Bauer, and Christopher D. Manning. 2025. A new pair of GloVes. *arXiv preprint arXiv:2507.18103*.

Surajit Chaudhuri, Venkatesh Ganti, and Raghav Kaushik. 2006. A primitive operator for similarity joins in data cleaning. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*.

Stanley F. Chen and Joshua Goodman. 1998. An empirical study of smoothing techniques for language modeling. Technical Report TR-10-98, Center for Research in Computing Technology, Harvard University.

Jasmine Collins, Jascha Sohl-Dickstein, and David Sussillo. 2017. Capacity and trainability in recurrent neural networks. In *International Conference on Learning Representations*.

Mostafa Dehghani, Anurag Arnab, Lucas Beyer, Ashish Vaswani, and Yi Tay. 2022. The efficiency misnomer. In *International Conference on Learning Representations*.

Thomas G. Dietterich. 1998. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. 2019. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2185–2194.

Jeffrey L. Elman. 1990. Finding structure in time. *Cognitive Science*, 14(2):179–211.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.

Kenneth Heafield, Ivan Pouzyrevsky, Jonathan H. Clark, and Philipp Koehn. 2013. Scalable modified Kneser-Ney language model estimation. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 690–696.

John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743.

Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation*, 9(8):1735–1780.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems 35*.

Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The curious case of neural text degeneration. In *International Conference on Learning Representations*.

Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952*.

Armand Joulin, Edouard Grave, Piotr Bojanowski, Matthijs Douze, Hervé Jégou, and Tomas Mikolov. 2016. FastText.zip: Compressing text classification models. *arXiv preprint arXiv:1612.03651*.

Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. Bag of tricks for efficient text classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 427–431.

Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. 2016. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*.

Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. 2015. An empirical exploration of recurrent network architectures. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 2342–2350.

Daniel Jurafsky and James H. Martin. 2026. Speech and language processing. 3rd edition draft, August 2026. https://web.stanford.edu/~jurafsky/slp3/.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray,

Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Yoon Kim. 2014. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1746–1751.

Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*.

Leslie Kish. 1965. *Survey Sampling*. John Wiley & Sons, New York.

Tom Kocmi and Ondřej Bojar. 2017. An exploration of word embedding initialization in deep-learning tasks. *arXiv preprint arXiv:1711.09160*.

Siwei Lai, Kang Liu, Shizhu He, and Jun Zhao. 2016. How to generate a good word embedding. *IEEE Intelligent Systems*, 31(6):5–14.

Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8424–8445.

Omer Levy, Yoav Goldberg, and Ido Dagan. 2015. Improving distributional similarity with lessons learned from word embeddings. *Transactions of the Association for Computational Linguistics*, 3:211–225.

Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. 2016. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 110–119.

Hong Liu, Sang Michael Xie, Zhiyuan Li, and Tengyu Ma. 2023. Same pre-training loss, better downstream: Implicit bias matters for language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *PMLR*.

Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. 2020. On the variance of the adaptive learning rate and beyond. In *International Conference on Learning Representations*.

Ilya Loshchilov and Frank Hutter. 2017. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150.

Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. 2018. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*.

R. Thomas McCoy, Paul Smolensky, Tal Linzen, Jianfeng Gao, and Asli Celikyilmaz. 2023. How much do language models copy from their training data? evaluating linguistic novelty in text generation using RAVEN. *Transactions of the Association for Computational Linguistics*, 11:652–670.

Stephen Merity, Nitish Shirish Keskar, and Richard Socher. 2018. Regularizing and optimizing LSTM language models. In *International Conference on Learning Representations*.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26.

Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 1310–1318.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Lutz Prechelt. 1998. Early stopping, but when? In *Neural Networks: Tricks of the Trade*, pages 55–69. Springer.

Ofir Press and Lior Wolf. 2017. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 157–163.

Alec Radford, Rafal Jozefowicz, and Ilya Sutskever. 2017. Learning to generate reviews and discovering sentiment. *arXiv preprint arXiv:1704.01444*.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language under-

standing by generative pre-training. Technical report, OpenAI.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. Technical report, OpenAI.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Radim Řehůřek and Petr Sojka. 2010. Software framework for topic modelling with large corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, pages 45–50.

Nils Reimers and Iryna Gurevych. 2017. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 338–348.

Haşim Sak, Andrew Senior, and Françoise Beaufays. 2014. Long short-term memory recurrent neural network architectures for large scale acoustic modeling. In *Proceedings of Interspeech 2014*, pages 338–342.

Robin M. Schmidt, Frank Schneider, and Philipp Hennig. 2021. Descending through a crowded valley: Benchmarking deep learning optimizers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9367–9376.

Christopher J. Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. 2019. Measuring the effects of data parallelism on neural network training. *Journal of Machine Learning Research*, 20(112):1–49.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008.

Sida Wang and Christopher D. Manning. 2012. Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 90–94.

Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. 2020. Neural text generation with unlikelihood training. In *International Conference on Learning Representations*.

John Wieting and Douwe Kiela. 2019. No training required: Exploring random encoders for sentence classification. In *International Conference on Learning Representations*.

Yuhuai Wu, Mengye Ren, Renjie Liao, and Roger Grosse. 2018. Understanding short-horizon bias in stochastic meta-optimization. In *International Conference on Learning Representations*.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. 2020. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10524–10533.

Zhilin Yang, Zihang Dai, Ruslan Salakhutdinov, and William W. Cohen. 2018. Breaking the softmax bottleneck: A high-rank RNN language model. In *International Conference on Learning Representations*.

Zi Yin and Yuanyuan Shen. 2018. On the dimensionality of word embedding. In *Advances in Neural Information Processing Systems*, volume 31.

Jingzhao Zhang, Sai Praneeth Karimireddy, Andreas Veit, Seungyeon Kim, Sashank J. Reddi, Sanjiv Kumar, and Suvrit Sra. 2020. Why are adaptive methods good for attention models? In *Advances in Neural Information Processing Systems 33*.

Doudou Zhou. 2026a. DSA4213 course notebooks (Lecture1_Demo, Lecture3_Demo). National University of Singapore.

Doudou Zhou. 2026b. Lecture 3: Neural language models and recurrent architectures. DSA4213 Natural Language Processing for Data Science, National University of Singapore.

## A Decision Record

Tables A1 and A2 list the main decisions, each chosen from options compared on evidence (my own measurements or published work) and recorded with its reasons before the results it affects, with three stated exceptions: the tuning-rule extension (Appendix C), the revision of the error categories after a first reading (Appendix F), and the framing question itself, which I chose after the Part I and II test results so that it would use all three parts. No setting was changed after seeing a test result.

| Decision | Chosen | Considered and rejected, and why | Evidence |
|---|---|---|---|
| Question | Does better next-word prediction give better sentiment features, and at what cost? | Model families alone (leaves Part II aside; at S the LSTM and Transformer are only 0.010 nats apart); fixed embeddings alone (leaves Part I aside); cost alone (the S models run within 6% of each other's speed). Only this question needs all three parts. | Brief, Experimental Focus; Radford et al. (2018); Liu et al. (2023) |
| Corpus | Amazon All_Beauty reviews, split by product | WikiText-2 (Wikipedia, inside GloVe's data); IMDB (its unlabelled reviews are not documented as disjoint by movie from its test set); AG News (news, like Gigaword); TinyStories (no labels); Gutenberg (labels to build); Movies & TV (25 times larger, too big for one laptop). | Table B1; Hou et al. (2024); Maas et al. (2011) |
| Tokens and vocabulary | Course tokenizer plus <num>; tokens seen 5+ times in training | Minimum count 10 (0.28 points more <unk>); 3, 2 or 1 (5,279, 11,557 or 35,175 more words for 0.15, 0.24 or 0.35 points less <unk>); splitting contractions (would change Part I's tokens for Part II's sake). | Figure B1d; gensim's default minimum (Řehůřek and Sojka, 2010) |
| Scoring | Each review alone from <bos>, <eos> a target, $L = 513$ | One running stream (context from another person's review); sliding windows (a window choice could move the Transformer's score). | Zhou (2026b, slide 29) |
| Fair comparison | One recipe; only the peak rate tuned, by one rule; cores within 10% in parameters (2.3% achieved) | Equal hidden units (unequal sizes); equal total parameters (dominated by the 3.4M word tables); tuning everything (budget). | Kaplan et al. (2020); Collins et al. (2017); Dodge et al. (2019) |
| Lengths | Remove only reviews with no word or over 512 tokens | A 3- or 5-word minimum: removes 8.7% of positive against 3.4% of negative reviews at 3 words, and short reviews are real reviews (7.2% of reviews, 0.4% of tokens). | Figure B1a |
| English only | fastText plus my own word score, matched to review length | fastText alone (only 11 of 25 sampled one- or two-word flags were foreign; curly apostrophes made English look Russian); the word score alone (inherits fastText's labels). | Table B3; hand checks, Table B4 |
| Cap per product | 90, the smallest cap reaching 95% of a product's information | No cap (the top 1% of products hold 28% of reviews); a fixed 30, 100 or 200 (no reason in this data); 90% of the information (43; changes more data). | Kish (1965); Figure B1c |
| Duplicates | Remove same-person copies and 14+ word copies by different people | A 7-word line (8 of 10 sampled removals were everyday sentences); removing every repeat (most short repeats are everyday sentences written independently). | Figure B1b; Lee et al. (2022) |
| $n$-gram model | Own modified Kneser-Ney 5-gram, plus the course's add-$k$ | KenLM (rejects <unk> in its input); add-$k$ alone (Kneser-Ney is better at every order from 2, by up to 0.17 nats). | Chen and Goodman (1998); App. C |
| Transformer | Pre-LN, sinusoidal positions | Post-LN (needs a long warm-up; mine is 4%); learned positions (51,300 more parameters, 16% above the RNN's core). | Xiong et al. (2020); Vaswani et al. (2017) |
| Context | Each model's natural context, plus $J$ by target position | One 4-token window for all (inputs the models never trained on; hides the difference under study); a context ablation (extra runs). | Kaplan et al. (2020, Fig. 7) |
| Decoding | Temperature sampling, one $T$ measured on validation (1.01); 22 tokens; median of 9 samples shown | The course's $T = 0.8$ (text 1.1 to 1.5 nats too predictable); greedy (loops); top-$k$ or nucleus (hide the tail that $J$ pays for); 30 tokens (no reason in this data); hand-picked samples. | Holtzman et al. (2020, Sec. 4.2); Figure 1c |
| Cost | Several indicators on one workload, GPU and CPU | One indicator (they can disagree); generation timing (my Transformer recomputes its prefix, so it would time my code, not the design). | Dehghani et al. (2022) |
| Learning curves | $x$-axis in training targets; validation and a fixed training sample at the same checks | Steps (not comparable); wall-clock time (mixes in speed); running batch loss (averaged over a changing model, a noisier gap). | Kaplan et al. (2020, Fig. 2) |

Table A1: Decision record, part 1: data, setup and Part I.

| Decision | Chosen | Considered and rejected, and why | Evidence |
|---|---|---|---|
| Part II model | LSTM-S | Transformer-S (0.010 nats behind on validation); the choice rule was fixed before the results. | Table 1 |
| Dimension | $d = 100$ | 300 (better analogies, little gain as model input, three times the table); 50 (below the PIP-loss optimum for skip-gram on Text8, a corpus of similar size). | Carlson et al. (2025); Lai et al. (2016); Yin and Shen (2018) |
| Own vectors | Skip-gram with Lecture 2's settings | CBOW; my own GloVe (needs its C tools; skip-gram is a robust baseline in controlled comparisons). | Levy et al. (2015); Mikolov et al. (2013) |
| Public vectors | GloVe 2024, Wikipedia and Gigaword | GloVe 2014 (covered 93.6% of the vocabulary against 96.9% in the vocabulary study; lacks newer beauty words). | Carlson et al. (2025) |
| Missing words | Contractions from their halves, tokenizer splits from their pieces, `<num>` from number vectors, others random at GloVe's variance | Zeros or one shared mean vector (many identical rows). | Kim (2014) |
| Scale | Own table scaled to GloVe's root mean square | Unscaled (with Adam, a fixed table's scale acts like a learning rate for the first layer); scaling changes no similarity. | Kingma and Ba (2015) |
| Seeds and rate | 3 seeds per setting, one rate for all (0.01), checked at 0.003, 0.03 and 0.1 | One seed (one score is not enough to compare); tuning each setting (more runs, unequal procedure). | Reimers and Gurevych (2017) |
| Extra runs | GloVe trainable; random table fixed | None (cannot separate trainability from GloVe's extra data). | Kim (2014); Kocmi and Bojar (2017) |
| Labels | 1–2 stars negative, 4–5 positive, 3 out, balanced | Keeping 3 stars (unclear label). | Maas et al. (2011) |
| Models in Part III | All 19, in groups that change one factor | Two models (their rankings agree by chance half the time); the three S models (design and $J$ mixed up). | Section 4 |
| Features | The 100-number pre-output vector, last word and average | Top hidden state (100 to 1,120 numbers; longer vectors classify better on their own); max pooling (nothing in my results pointed to it). | Wieting and Kiela (2019, Sec. 4.1); Part I position results |
| References | Untrained copies, averaged tables, word counts, length | | Wieting and Kiela (2019); Wang and Manning (2012) |
| Classifier | Logistic regression, $C$ by 5-fold cross-validation split by product | A small MLP (less selective probe); a linear SVM (also linear, no probabilities); one fixed $C$ (unfair between 100 and 153,041 features); random folds (one product on both sides, so product cues leak into the choice of $C$). | Hewitt and Liang (2019); Radford et al. (2017) |
| Statistics | 51 McNemar tests fixed in advance, Holm-corrected; Kendall's $\tau$ over 13 models | All pairs (about 1,800 tests; a correction that hides real gaps); Pearson's $r$ (one extreme model can drive it); seed repeats in the ranking (counts one setting three times). | Dietterich (1998); Holm (1979) |
| Error analysis | Groups over all test reviews, plus 40 random mistakes in categories fixed before reading (revised after a first reading, App. F) | Hand-picked examples (not representative); reading all mistakes (about 1,600). | Socher et al. (2013) |

Table A2: Decision record, part 2: Parts II and III.

## B Data and Preprocessing Evidence

**Choice of corpus.** Table B1 compares the seven corpora I considered. WikiText-2 is a standard language-modelling benchmark, but it is Wikipedia text, which GloVe was trained on, so Part II's public vectors would have seen it. IMDB has sentiment labels and little overlap with GloVe, but its 50,000 unlabelled reviews, the natural language-model training text, are not documented as disjoint by movie from its labelled test set (Maas et al., 2011). Amazon Movies & TV has the same properties as All_Beauty with about 25 times as many reviews; as the assignment allows, I took the category whose size suits one laptop.

**Cleaning.** Before tokenizing, Amazon video tags (in 5,951 reviews) become spaces, links to other products keep the product name (474 reviews), e-mail addresses are removed as personal data (42 reviews), "&" becomes "and" and curly apostrophes are straightened. The course tokenizer is the regular expression `[a-z]+(?:'[a-z]+)?|[.,!?;:]` on lowercased text, extended to keep `<num>` whole.

**Lengths.** Reviews under three words are 7.2% of reviews but hold 0.4% of tokens, and a minimum length would change the label mix (Figure B1a), so they stay. The 512-token limit removes 723 reviews (0.10%) and lets one Transformer context of 513 positions hold every review with its `<bos>`.

**English only.** fastText's language identifier (Joulin et al., 2017, 2016), its compressed lid.176 model used through the fast-langdetect package, would on its own remove 15,434 reviews, a third of them of one or two words, where its guesses are least reliable: of 25 one- or two-word reviews it labelled non-English with probability below 0.5, only 11 were foreign. Curly apostrophes also made it label English reviews as Russian or Thai, so they are straightened first. My word score sums, over a review's words, the log of how much more often each word appears in non-English than in English reviews (a score of 2 means about 7 times). It is learnt from reviews of seven or more words that fastText labels with probability 0.9 or more (516,880 English, 5,090 other), so it inherits fastText's labels. Reviews of up to six words are removed when their score is 2 or more. Longer reviews are removed when fastText's top language is not English and either it is at least 90% sure or the score is 2 or more, and also when a sentence of five or more words is non-English for fastText (probability 0.9 or more) with an average word score of 2 or more, as in an English review followed by a translation. The rule keeps 3,997 reviews that fastText flagged, almost all of them short (Table B3). Filtering a language-model corpus by language and duplicates follows C4 (Raffel et al., 2020).

| Corpus | Labels | Item split | Not in GloVe |
|---|---|---|---|
| WikiText-2 | none | – | no |
| AG News | topics | no | partly |
| TinyStories | none | – | yes |
| Gutenberg novels | to build | yes | yes |
| IMDB | sentiment | partly | yes |
| Amazon Movies & TV | stars | yes | yes |
| Amazon All_Beauty | stars | yes | yes |

Table B1: Candidate corpora. Labels: in-domain labels for Part III (Gutenberg's, such as authorship, would have to be built). Item split: all text about one product, movie or book can be kept on one side of every split, so no Part III test item appears in language-model training (– where there are no labels). Not in GloVe: the text is unlikely to be in GloVe's Wikipedia and Gigaword training data (AG News is news, like Gigaword).

| Corpus step | Reviews | Share |
|---|---|---|
| Raw All_Beauty reviews | 701,528 | |
| – no word, or over 512 tokens | 2,583 | 0.4% |
| – not English | 11,634 | 1.7% |
| – duplicates | 8,209 | 1.2% |
| – over 90 per product | 91,969 | 13.5% |
| Clean corpus | 587,133 | 83.7% |
| **Pile (split keys)** | **Reviews** | **Size** |
| LM training $[0, 0.5)$ | 295,645 | 10.96M targets |
| LM validation $[0.5, 0.55)$ | 28,505 | 1.07M targets |
| LM test $[0.55, 0.6)$ | 29,827 | 1.12M targets |
| Part III test $[0.6, 0.7)$ | 57,288 | 24,362 bal. |
| Part III training $[0.7, 1)$ | 175,868 | 74,344 bal. |

Table B2: From raw reviews to piles. Shares are of the reviews entering each step; targets are tokens plus `<eos>`; Part III sets balance 1–2 star (negative) and 4–5 star (positive) reviews.

**Duplicates.** Exact copies by the same person are one review counted again (8,021 removed), and near copies by the same person share 90% or more of their distinct words (69 removed). A first line of 7 words for exact copies by different people removed mostly everyday sentences (8 of 10 in a hand-checked sample), so it moved to 14: on the final corpus, near-identical but not identical reviews by different people occur at every length from 7 to 13 words and never from 14 to 26 (Figure B1b). Exact copies of 14 or more words by different people are removed (119).

**Cap per product.** In the raw data, the top 1% of products held 28% of reviews and one product had 1,962. Star ratings of one product correlate, with intraclass correlation $\rho = 0.1745$. By Kish's design

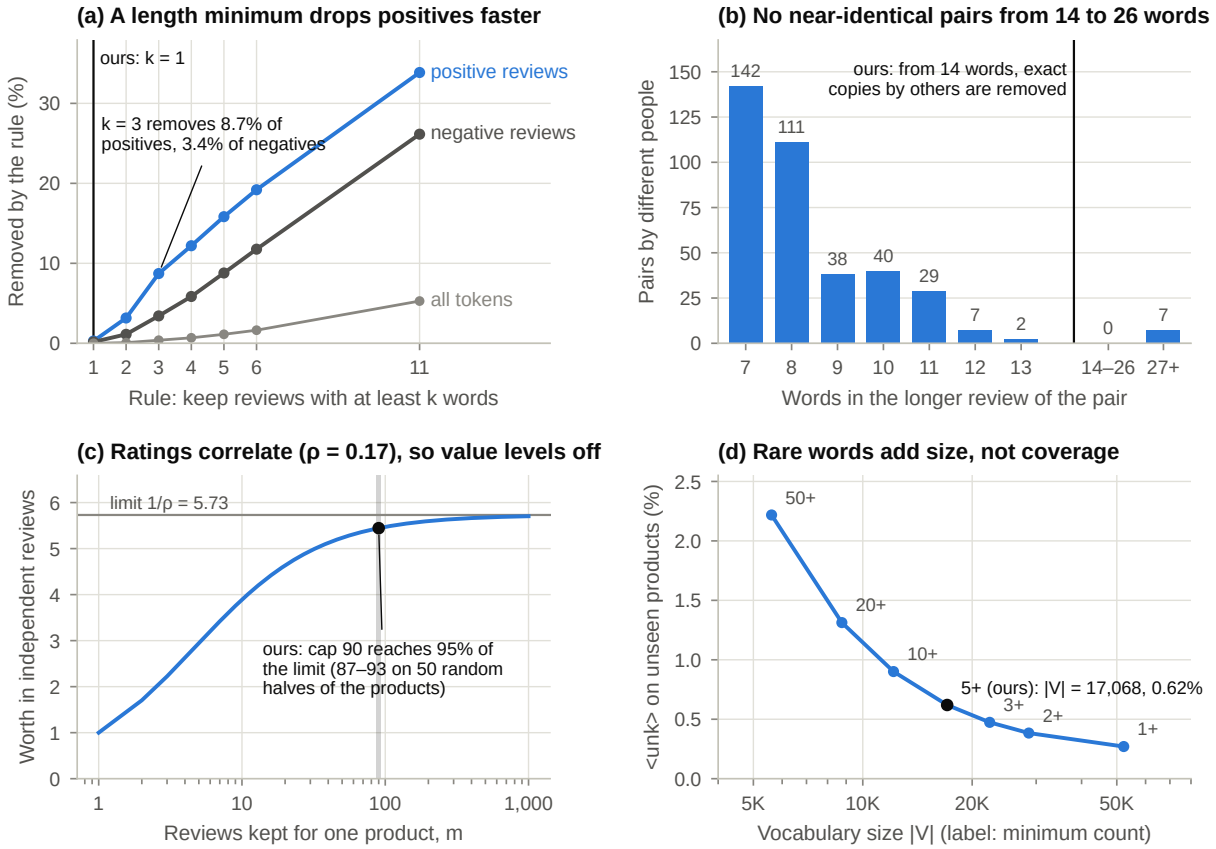



Figure B1: The measurement behind four preprocessing thresholds; black marks my choice. (a) Share of positive (4–5 stars) and negative (1–2 stars) reviews, and of all tokens, that a minimum of $k$ words would remove (raw reviews after cleaning). I only remove reviews with no word ($k = 1$). (b) Pairs of near-identical but not identical reviews by different people, sharing 90% or more of their distinct words, by the length of the longer review (final corpus, exact search; Chaudhuri et al., 2006; Bayardo et al., 2007). Below 14 words such pairs are everyday sentences, and exact copies of them are kept; from 14 words, exact copies by different people are removed. The 7 pairs of 27 or more words read as copies and remain (Table B6). (c) Kish's design effect with the intraclass correlation of star ratings within a product, $\rho = 0.1745$: $m$ reviews are worth $m/(1 + (m - 1)\rho)$ independent ones, approaching $1/\rho = 5.73$. A cap of 90 reaches 95%; the grey band is the cap's range (87 to 93) when $\rho$ is re-estimated on 50 random halves of the products. (d) Vocabulary built on the products with keys below 0.5 (the later training pile) and tested on the rest; labels give the minimum count.

| Words | Reviews | fastText | Our rule | Flags kept |
|---|---|---|---|---|
| 1–2 | 48,903 | 4,953 | 2,608 | 2,546 |
| 3–6 | 89,885 | 4,192 | 2,945 | 1,271 |
| 7–20 | 226,260 | 4,446 | 4,270 | 177 |
| 21+ | 333,894 | 1,843 | 1,849 | 3 |
| All | 698,942 | 15,434 | 11,672 | 3,997 |

Table B3: Reviews removed as not English, by length in words: by fastText's top guess alone and by my rule; flags kept are fastText flags that my rule keeps. From the run that designed the rule (698,942 reviews, before the final cleaning step); the final pipeline removed 11,634 (Table B2).

effect, $m$ reviews of a product estimate its mean rating as precisely as $m/(1 + (m - 1)\rho)$ independent reviews would, which approaches $1/\rho = 5.73$; 90 is the smallest $m$ that reaches 95% of that limit. Re-estimated on 50 random halves of the products, $\rho$ ranges from 0.170 to 0.181 and the cap from 87 to 93 (Figure B1c). Kept reviews are chosen by a fixed random number per review (seed 4213).

**Pile sizes.** Pile sizes come from 20 slices of 5% of the split keys. On every slice a unigram model's $J$ has a standard error of 0.0030 to 0.0035 nats when reviews are resampled, so 5% is enough for validation and for test; this ignores the correlation between reviews of one product and the variation between training seeds. Training must cover $[0, 0.5)$ because the pilot run used products from that range. Larger training piles would lower every model's $J$ somewhat but shrink Part III's balanced training set from 74,344 reviews to about 49,700 (training on $[0, 0.6)$, Part III training on $[0.7, 0.9)$) or 24,700 (training on $[0, 0.7)$, Part III training on $[0.9, 1)$); how the ranking changes with more training data is not tested.

| # | Sample | $n$ | Author | Claude |
|---|---|---|---|---|
| 1 | Removed, 1–6 words | 40 | 40 | 40 |
| 2 | Removed, 7+ words | 40 | 40 | 40 |
| 3 | Removed, foreign sentence | 28 | 28 | 27 |
| 4 | Kept, 1–6 words, flagged | 30 | 30 | 30 |
| 5 | Kept, 7+ words, flagged | 30 | 29 | 29 |
| 6 | Removed, own near copy | 10 | 10 | 10 |
| 7 | Removed, 14+ word copy by other | 10 | 10 | 10 |
| 8 | Kept, 7–13 word copy by other | 10 | 10 | 10 |

Table B4: Hand checks: reviews the rule got right in a random sample of $n$ (sample 3 holds all 28 such removals). Flagged: fastText's top guess was not English. Own near copy: by the same person; by other: by a different person. Right means not English (1, 2); contains a non-English sentence (3); English (4, 5); the same review again (6); copied, not written independently (7); an everyday sentence anyone could write (8).

**Vocabulary.** The cut-off was measured by building a vocabulary on the products with keys below 0.5, which later became the training pile, and counting <unk> on the other half; swapping the halves gives the same curve (Figure B1d). Lowering the minimum count from 10 to 5 adds 4,918 words and removes 0.28 percentage points of <unk>; lowering it further to 3, 2 and 1 adds 5,279, 6,278 and 23,618 words for 0.15, 0.09 and 0.11 points. Five is also gensim word2vec's default minimum count (Řehůřek and Sojka, 2010), so Part II's own embeddings see every vocabulary word at least five times.

**Hand checks.** I read random samples of each rule's decisions (seed 4213) and counted those the rule got right (Table B4). Claude, the AI assistant used in this project (Appendix G), counted the same samples. The counts agree on 7 of 8 samples. For samples 1 to 6 I had seen Claude's counts before counting; for 7 and 8 I counted first. The one disagreement is a review whose only non-English words are a five-word holiday greeting: I counted it as containing a foreign sentence, Claude did not. Samples of 10 are small: 10 of 10 right still allows an error rate of up to 26% (one-sided 95% bound).

**Checks on the final corpus.** A script reruns these checks on the file the models read (Table B5). The corpus fingerprint (SHA-256 of the uncompressed file) starts `16ec4ec1`, so anyone who rebuilds the corpus can confirm it is identical.

**Limitations.** Each limitation of the preprocessing is measured (Table B6). Every filter was designed on the whole corpus before the split, and none used a model result, but some used text or ratings of products that later became evaluation

| Check | Result |
|---|---|
| Rows, tokens, fingerprint match the report | yes |
| A rerun gives a byte-identical file | yes |
| Tokens outside the tokenizer's pattern | 0 |
| Review length | 1 to 512 tokens |
| Reviews per product | at most 90 |
| Products with more than one split key | 0 |
| Products per tenth of the keys | 11,000 to 11,237 |
| Same-person repeats | 0 |
| 14+ word copies by different people | 0 |
| E-mail addresses left | 0 |
| Reviews with 4+ foreign function words | 2 |

Table B5: Checks on the clean corpus; every rule holds. The last row counts leftovers that remain as a limitation.

| Limitation | Share | Consequence |
|---|---|---|
| Numbers become <num> | 15.9% | "0 stars" and "5 stars" look alike |
| Reviews by repeat users | 14.5% | a user's style may reach several piles |
| Short repeats kept | 7.4% | a short review can be in training and test |
| Runs of punctuation | 2.2% | 0.34% of tokens, the same for every model |
| Emoji dropped | 2.0% | some sentiment signal lost |
| Reviews over 512 tokens | 0.10% | 2.0% of tokens removed |
| Contractions split | 0.06% | odd apostrophes give "don t" |
| Accented words split | $\geq$0.05% | "crème" becomes "cr me" |
| Long near copies | 7 pairs | 2 of them span two products |

Table B6: Measured limitations of the preprocessing. Shares are of reviews in the clean corpus, except numbers, emoji and long reviews (raw reviews).

products: the length rule compared the label mix of all reviews, the word score was learnt from all reviews, the cap's $\rho$ uses every product's ratings, and the vocabulary cut-off was compared on the other half of the products. These choices barely depend on which products became test products: the cap moves only between 87 and 93 across random halves of the products, and swapping the vocabulary halves gives the same curve.

## C Model and Training Evidence

**Pilot run.** Before the real runs, a 28-minute pilot run trained each candidate model for 300 steps (about 0.2 epochs) at 8,192 positions per batch and a peak learning rate of $10^{-3}$, on products with keys below 0.45 and with its own vocabulary ($|V| = 16{,}270$); it validated on keys 0.45 to 0.5. It timed 10 steps at five batch sizes, logged unclipped gradient norms, and estimated the gradient noise scale $B_{\text{simple}}$ after 50, 150 and 300 steps from 8 micro-batches (McCandlish et al., 2018, App. A.1). It set three values (Table C1, Figure C1). *Batch size:*

| Model | Core | K targets/s 8K | K targets/s 16K | Memory 8K (GiB) | Noise scale | Mean norm | Clipped steps | $J$ after 300 steps |
|---|---|---|---|---|---|---|---|---|
| RNN-S | 365,668 | 44.0 | 43.2 | 3.9 | 13,095 | 0.96 | 23% | 5.726 |
| LSTM-S | 372,100 | 44.2 | 43.4 | 3.9 | 8,060 | 0.71 | 11% | 5.812 |
| Transformer-S | 374,200 | 40.6 | 40.2 | 4.0 | 23,909 | 0.67 | 11% | 5.028 |
| RNN-M | 1,480,740 | 34.8 | 37.9 | 3.8 | 15,407 | 1.21 | 41% | 5.718 |
| LSTM-M | 1,489,500 | 37.6 | 37.1 | 4.1 | 10,480 | 0.87 | 17% | 5.699 |
| Transformer-M | 1,578,980 | 32.5 | 32.0 | 4.8 | 11,722 | 0.76 | 11% | 4.963 |

Table C1: Pilot run: core parameters, training speed at 8K and 16K positions per batch, peak memory at 8K, gradient noise scale $B_{\text{simple}}$ at step 300 (targets), mean unclipped gradient norm and share of clipped steps over the second half of the run, and validation $J$ after 300 steps. S models are the main size; M models are the size check, with cores matched within 6.6%.

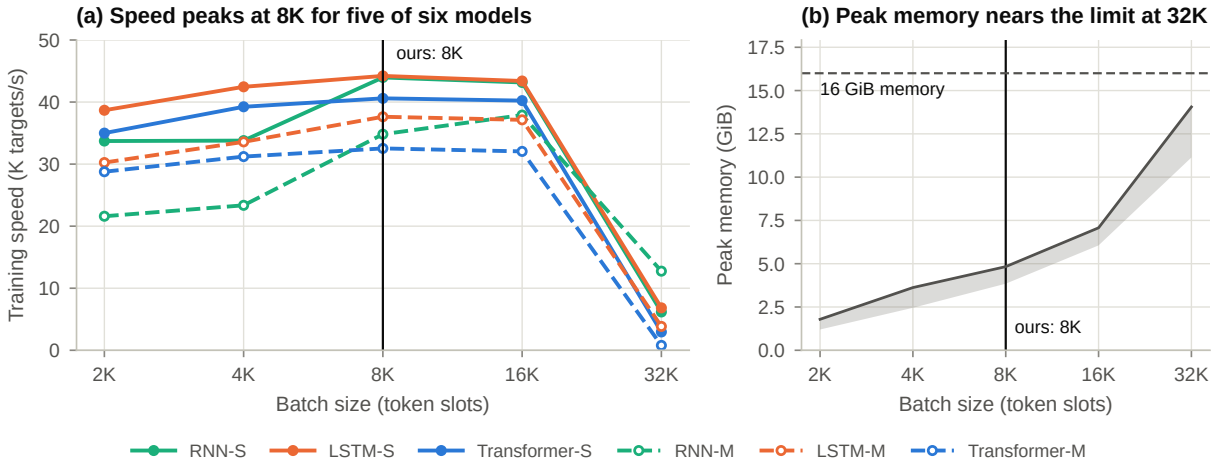



Figure C1: Pilot-run speed and memory by batch size. (a) Training speed of the six candidate models; solid lines are the main size (S), dashed lines the size check (M). (b) Peak memory over the six models: the band is their range and the line the largest. I use 8K positions per batch: speed rises up to 8K for every model and is flat or lower at 16K for five of six, and 8K stays below every step-300 estimate of the gradient noise scale.

speed rises up to 8K positions for every model and is flat or lower at 16K for five of six (RNN-M gains 9%); at 32K, memory reaches 11.1 to 14.1 GiB of 16 GiB and speed collapses. At step 300, $B_{\text{simple}}$ is at least 8,060 targets for every model, while an 8K batch holds about 7,150 targets and a 16K batch about 14,000, so 8K is the largest size below every step-300 estimate. The estimates are noisy: at step 150, RNN-M's was 5,977, and LSTM-S's was undefined because its estimate of the squared gradient norm $|G|^2$ came out negative. The throughput plateau therefore decides, and the noise scale only supports it. *Clipping:* 1.0 lies within half to ten times every model's mean gradient norm over the second half of the run, 0.67 to 1.21 (Pascanu et al., 2013). The vanilla RNNs clip most often (23% and 41% of late steps) because their typical norm is near 1.0 (late means 0.96 and 1.21). Adam scales each step by a running average of gradient size, so I expect clipping to change little beyond trimming the largest steps; this is not tested. *Validation cost:* one validation pass took 8 to 10 s. Evaluation time, including the training sample scored at each check, is logged separately and left out of training time.

**$n$-gram models.** Kneser-Ney uses the closed-form discounts of Chen and Goodman (1998, eq. 26) for counts of 1, 2, and 3 or more, with continuation counts for the lower orders; $n$-grams that start with <bos> keep their raw counts, as in Heafield et al. (2013, 3.2), because no word can come before <bos>; a discount outside its valid range falls back to 0.5, 1 or 1.5, as in KenLM's discount fallback. Add-$k$ interpolates add-$k$ estimates of every order up to $n$. Its $k$ is searched over powers of ten from $10^{-4}$ to 1, and the grid is extended tenfold while the best value is at an edge; its weights are fit by EM on validation data (Jurafsky and Martin, 2026), so its validation $J$ is slightly optimistic, but its test $J$ is not. Both choose $n = 5$ on validation, where Kneser-Ney beats order 4 by only 0.003 nats, because 79% of validation 5-grams never occur in training. I did not use KenLM, which rejects <unk> in its input. Tests check that every distribution sums to 1 and that my Kneser-Ney matches a slow reference implementation for

orders 1 to 4.

**Recipe and sources.** Table C2 lists every setting with its evidence. The pilot run checked three of them on this data (batch size, clipping and validation cost); the others follow the cited sources, most of which study larger Transformer models with subword tokens.

**Rules fixed before any result.** Three rules were written down before the first run, so the results cannot steer them.

1. *Overfitting.* A model overfits if its validation $J$ stops improving while its training-sample $J$ keeps falling, before the last quarter of its planned run. If any model does, all three get a run with dropout 0.1 on the input embeddings and before the output layer, at their chosen learning rates; if that lowers any model's validation $J$, all three get one more run at 0.3. Main results stay at dropout 0. *Outcome:* no run met the rule (every kept 6-epoch run's best checkpoint is one of its last two checks), so no dropout runs were made.

2. *Training budget.* If a model's validation $J$ still falls in its last epoch by more than the gap to its neighbour in the ranking, all three models get one longer run (10 epochs) at their chosen rates, and both budgets are reported. *Outcome:* the LSTM's validation $J$ fell by 0.023 in its last epoch against a gap of 0.010 to the Transformer, so all three got 10-epoch runs (Table 1).

3. *Short tries.* The validation checks show which rate one-epoch tries would have chosen, and whether it matches the full-run winner. Short tries can mislead either way: short horizons favour small step sizes (Wu et al., 2018), while the best learning rate falls as training gets longer (Bjorck et al., 2025). *Outcome:* validation $J$ after one epoch picks the same rate as the full tries for all six models.

**Change after the first results.** The tuning rule fixed before the runs allowed one step past an edge. The LSTM's best rate was then the largest it tried (0.01, after 0.0003, 0.001 and 0.003), so before the Transformer's and RNN's tries finished, I extended the rule for every model: keep stepping half a decade outward while the best rate stays at the edge, as Bengio (2012, 3.3.1) recommends. The extension tried 0.03 for the LSTM-S and LSTM-M, which was worse in both cases, so 0.01 stayed; every other model's best rate was already inside its tries (Table C3).

**Limitations of the recipe.** The shared settings may suit one model better than another, since only the learning rate is tuned per model. A few tries find a good learning rate, not always the best, and the winner can depend on the tuning budget (Dodge et al., 2019), so I report the tries. AdamW may slightly hold back the RNN and LSTM: plain SGD has been reported to work better for LSTM language models (Merity et al., 2018), though that paper does not test Adam itself. The 100 numbers that feed each output layer limit the rank of every model's score matrix (Yang et al., 2018), equally for all three, which may shrink the gaps between them; tying $E$ to the output layer would improve perplexity (Press and Wolf, 2017) but would freeze the output layer with $E$ in Part II.

| Setting | Value | Evidence |
|---|---|---|
| Tuning | Peak learning rate per model, starting from $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$ and stepping half a decade further while the best is the largest or smallest tried (range $10^{-5}$ to 0.1); every try a complete planned run; best try kept | If only one hyperparameter can be tuned, tune the learning rate (Goodfellow et al., 2016); log-spaced values, and explore beyond the border when the best value lies on it (Bengio, 2012, 3.1.1, 3.3.1); which model looks best depends on the tuning budget, so every model gets the same search rule and the tries are reported (Dodge et al., 2019) |
| Optimizer | AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$; weight decay 0.1 on weight matrices, none on biases, LayerNorm or $E$ | Adam's suggested defaults (Kingma and Ba, 2015); SGD converges much more slowly on attention models (Zhang et al., 2020); no newer optimizer consistently beats Adam (Schmidt et al., 2021); decoupled decay (Loshchilov and Hutter, 2019); 0.1 as in GPT-3 (Brown et al., 2020) |
| Batch | 8,192 positions including padding (about 7,000 targets in the real runs) for every model | Speed peaks at 8K for five of six models (Figure C1); below every step-300 noise-scale estimate (McCandlish et al., 2018); one size for all, because the best learning rate changes with the batch size (Shallue et al., 2019) |
| Schedule | Linear warm-up over 4% of steps, then one cosine curve to 10% of the peak at the planned last step | Adam's early steps have high variance (Liu et al., 2020); 4,000 of 100,000 steps in Vaswani et al. (2017); cosine form of Loshchilov and Hutter (2017); overshooting the run by more than 25% hurts, and decaying 10 times is slightly better than decaying to 0 (Hoffmann et al., 2022) |
| Clipping | Global gradient norm 1.0 | Half to ten times the mean norm can still converge (Pascanu et al., 2013); pilot-run means 0.67 to 1.21 (Table C1); GPT-3 also clips at 1.0 (Brown et al., 2020) |
| Stopping | Validation $J$ 4 times per epoch; stop after 3 checks without a new best; keep the best checkpoint | Patience-based early stopping (Goodfellow et al., 2016, Alg. 7.1); validation curves almost always have more than one local minimum (Prechelt, 1998) |
| Run length | 6 epochs (about 66M targets) per run, for every model and try | The longest run that lets all three models finish their tries overnight on my machine; the course demos also train for a fixed number of epochs (6 in Lecture3_Demo); rule 2 below tests the budget |
| Dropout | 0 in every model | 0.35 parameters (word tables included) per training target, against 26 and 16 for AWD-LSTM on Penn Treebank and WikiText-2 (Merity et al., 2018, 2017); overfitting is measured, rule 1 below |
| Model details | Whole reviews, no truncated backpropagation; one recurrent layer; LSTM forget-gate bias 1; Transformer: ReLU feed-forward, final LayerNorm, token vectors scaled by $\sqrt{d_{\text{model}}}$; $E$ uniform in $\pm 0.1$ with a zero <pad> row | Forget-gate bias (Jozefowicz et al., 2015); Transformer as in Vaswani et al. (2017) with pre-LN (Xiong et al., 2020), as in GPT-2 (Radford et al., 2019); projection before the output layer (Sak et al., 2014; Jozefowicz et al., 2016) |
| Seed | 4213 for initialisation, data order and samples | |

Table C2: The shared training recipe of the three neural models, with its evidence.

| Model | $3\times10^{-4}$ | $10^{-3}$ | $3\times10^{-3}$ | $10^{-2}$ | $3\times10^{-2}$ |
|---|---|---|---|---|---|
| RNN-S | 4.409 | 4.069 | **3.915** | 4.537 | |
| LSTM-S | 4.627 | 4.130 | 3.845 | **3.712** | 3.726 |
| Transformer-S | 4.091 | 3.813 | **3.721** | 3.728 | |
| RNN-M | 4.341 | **4.003** | 4.082 | | |
| LSTM-M | 4.504 | 3.991 | 3.728 | **3.606** | 3.635 |
| Transformer-M | 3.894 | 3.646 | **3.573** | 3.640 | |

Table C3: Validation $J$ of every learning-rate try (each a full 6-epoch run); the best in bold.

## D Additional Part I Results

**Generated text, intervals.** 95% bootstrap intervals over the 300 openings (1,000 resamples) are in the result file `generation.json`; for example LSTM-S's share of repeated 4-grams is 0.2 to 1.1% and its copied 5-grams 13.2 to 16.2%. Distinct-$n$ is not an average over samples, so it gets split-half ranges instead. Table D3 shows the median sample of all eight generating models for all six prompts; the full sets of nine samples stay on my machine, outside the repository, like all other generated and review text.

**Memory.** I do not report GPU memory for the main runs: the reading on Apple's MPS is the driver's total including its cache, so it measures the allocator as much as the model (the pilot-run readings in Table C1 carry the same caveat). Every timed pass in Table 1 reproduced the test $J$, and Table D1 lists the position results behind Figure D1.

## E Additional Part II Details

**Coverage and missing words.** Of the 17,064 vocabulary words, 16,256 (95.3%) have an ex-

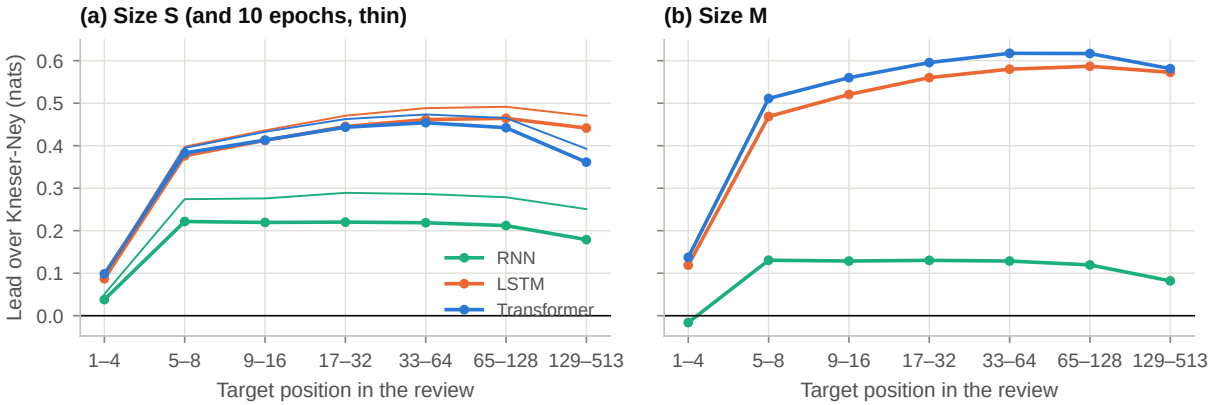



Figure D1: Each model's lead over the Kneser-Ney 5-gram by target position (its $J$ minus the model's, test pile). On targets 1 to 4 the 5-gram sees the whole history, so all models have the same information. Thin lines in (a): the 10-epoch runs.

| Targets at position | 1–4 | 5–8 | 9–16 | 17–32 | 33–64 | 65–128 | 129–513 |
|---|---|---|---|---|---|---|---|
| Test targets | 117,182 | 104,739 | 173,277 | 235,331 | 234,154 | 158,869 | 91,658 |
| Kneser-Ney $J$ | 3.870 | 3.992 | 4.044 | 4.093 | 4.171 | 4.243 | 4.335 |
| *Lead over Kneser-Ney (its $J$ minus the model's)* | | | | | | | |
| RNN-S | 0.038 | 0.222 | 0.219 | 0.220 | 0.219 | 0.212 | 0.179 |
| LSTM-S | 0.087 | 0.376 | 0.413 | 0.445 | 0.461 | 0.464 | 0.441 |
| Transformer-S | 0.098 | 0.383 | 0.413 | 0.443 | 0.454 | 0.442 | 0.361 |
| RNN-M | −0.016 | 0.130 | 0.129 | 0.130 | 0.129 | 0.120 | 0.082 |
| LSTM-M | 0.119 | 0.469 | 0.521 | 0.560 | 0.580 | 0.587 | 0.573 |
| Transformer-M | 0.137 | 0.511 | 0.560 | 0.596 | 0.617 | 0.617 | 0.581 |
| RNN-S, 10 ep. | 0.050 | 0.274 | 0.276 | 0.289 | 0.286 | 0.279 | 0.251 |
| LSTM-S, 10 ep. | 0.096 | 0.398 | 0.436 | 0.470 | 0.488 | 0.492 | 0.470 |
| Transformer-S, 10 ep. | 0.095 | 0.395 | 0.433 | 0.463 | 0.474 | 0.465 | 0.393 |

Table D1: Test $J$ by target position: the Kneser-Ney 5-gram's own $J$ and each neural model's lead over it (nats; positive means the neural model is better).

| Text | Rep-4 | Dist-2 | Copied | Gap |
|---|---|---|---|---|
| Real continuation | 0.0% | 0.702 | 19.7% | |
| Kneser-Ney 5-gram | 0.2% | 0.725 | 23.0% | −0.07 |
| RNN-S | 0.3% | 0.676 | 13.5% | +0.02 |
| LSTM-S | 0.6% | 0.675 | 14.7% | +0.05 |
| Transformer-S | 0.4% | 0.680 | 15.3% | −0.08 |
| Own skip-gram, frozen | 0.2% | 0.698 | 11.7% | +0.19 |
| GloVe, frozen | 0.2% | 0.717 | 8.9% | +0.29 |
| GloVe, trainable | 0.5% | 0.694 | 14.0% | +0.09 |
| Random, frozen | 0.2% | 0.702 | 10.4% | +0.19 |

Table D2: Generated text at $T = 1.01$, 300 test openings (5 tokens, then 22 new; <eos> blocked). Rep-4: repeated 4-grams (Welleck et al., 2020). Dist-2: distinct bigrams (Li et al., 2016). Copied: 5-grams found in the training pile (McCoy et al., 2023). Gap: the model's $J$ on its own text minus its $J$ on the real text (nats).

act GloVe vector, 279 contractions are built from their halves (GloVe's tokenizer splits "don't" into "do" and "n't"), and 9 words that CoreNLP splits, such as "cannot" and "gonna", from their pieces; <num> takes the average of GloVe's number vectors among its 10,000 most frequent words. The other 522 rows, 519 rare words such as "reccomend" and "polygel" plus <bos>, <unk> and <eos>, are random with the mean square of GloVe's exact rows (Kim, 2014); <pad> is zero. The frozen random table has the same root mean square (0.541), about nine times the trainable table's starting scale (0.058), so the 0.319-nat cost of freezing it mixes the loss of trainability with a change of scale. My skip-gram table (gensim, window 5, 5 negative samples, 15 epochs, $d = 100$, one thread, seed 4213) is multiplied by 1.69 so its root mean square matches GloVe's (0.541).

**Learning-rate check.** At seed 4213 the fixed tables were also trained at 0.003, 0.03 and 0.1. My vectors reach validation $J$ 3.965, 3.815, 3.800 and 4.532 at 0.003, 0.01, 0.03 and 0.1, GloVe 4.099, 3.976, 3.919 and 4.805; both 0.1 runs were unstable and stopped early. The trainable random table is best at 0.01 (0.03: 3.726). The order of the settings is the same at each setting's best rate, and the main comparison stays at 0.01, as fixed before the runs.

**Test $J$ per seed.** For seeds 4213, 4214 and 4215, the random trainable table reaches 3.708, 3.703 and 3.704, my frozen vectors 3.811, 3.795 and 3.810, and frozen GloVe 3.973, 3.959 and 3.954. Freezing therefore costs 0.103, 0.091 and 0.107 nats with

| | |
|---|---|
| Prompt 1: (no prompt: the start of a review) | |
| KN 5-gram | these are great for long and thick beards. i'm trying to hold them down later in beauty needle like a good |
| RNN-S | tangled and tangles feeling no mess. waste of effort. so had to dry them out. cause its adjusted to |
| LSTM-S | v en i have trouble damaged hair, this comb is safe, convenient and convenient to use just for traveling or |
| Transformer-S | this professional maintenance product is so pretty. love it and washing blow outs so much so i wash it off quickly |
| Own, frozen | sent out excellent german after shipping same day shoulder out of length!!!! thank you so much!! |
| GloVe, frozen | all the brushes were soft. good news. did not work but when u rock without frying tips, it was |
| GloVe, trainable | it's a really cute wig. very wearable. no complaints. great wig box multiple items except if you have it |
| Random, frozen | moisture. easy to apply and easy to use applied. use it on my lips, out machine safe; like |
| Prompt 2: *this shampoo …* | |
| KN 5-gram | makes great hold of lotion. it is very thick and one other messing with four heads are blue wings and obviously |
| RNN-S | conditioner smells like baby oils. it really does what it left on which i felt like it has contains a patchouli |
| LSTM-S | received was advised i was receiving my hair in conditioner and cucumber soap. i wash my hair right before using it |
| Transformer-S | second conditioner is easy to use and no shedding. it has good vegan do not break out or pungent or chemically |
| Own, frozen | feels so gentle on my scalp. so much i bought <num> less to none used to one wash. it also |
| GloVe, frozen | makes my fav! the pure ingredients and for moisture, sulfates, and works fine for my neck, <num> years |
| GloVe, trainable | is wonderful and expensive for me. i wish there was better coverage though then that. i put two strokes and |
| Random, frozen | has a temperature its smell gone but it could be better for my money or child. i share like that and |
| Prompt 3: *i was really disappointed with this …* | |
| KN 5-gram | product. it's so hard to put them back a fuller, healthier and travel to a more natural well this vendor |
| RNN-S | product. there are not as reasonably priced... ugh? now? one of the girls like packaging and |
| LSTM-S | product at all times it caused this moisturizer that felt greasy and could not feel to. i didn't even care for |
| Transformer-S | shampoo i have been telling me i wanted to try its authenticity of this product, this was up to ease my |
| Own, frozen | product only if i'm getting ink to charge with water but this was so super easy. i did not care with |
| GloVe, frozen | product. was hard but it did my girls hair the front of the upcoming! i don't not stop my hair |
| GloVe, trainable | product. not long lasting and had a brighter look. don't try it yet. i don't have white or anything |
| Random, frozen | brush! so much better than the granddaughters drawer. i've used tea brushes for a little a few years and gel |
| Prompt 4: *the color is beautiful but …* | |
| KN 5-gram | it is buildable though a few days which i have purchased there is black with extra prongs for those times when you |
| RNN-S | the hair i received was quite natural. it did shed a bit! no one could tie it and check when |
| LSTM-S | very vibrant to have for deep color. it does come in a dark black light and a little dark larger than |
| Transformer-S | it is blotchy when applied. i can't color it as much and even other shades available. this product does not |
| Own, frozen | i dont feel like it was garbage together even but still i wasn't sure thinking how to do it would update it |
| GloVe, frozen | i can't wish it's one color but the only thing i can't call to, everytime with this lip stick. i |
| GloVe, trainable | i guess not for me. lot cheaper than i wanted. however, oh note, i like this bundle. |
| Random, frozen | the hair i ever did not wear a lot of strands a thin or glued. it clumps, and twist around |
| Prompt 5: *the scent of this perfume is …* | |
| KN 5-gram | nice. adjustable so you really want it lifts and has an <unk> stay looking fresh for <num> line was off. |
| RNN-S | very aromatic and floral. it works nicely for me, as a mix it is for you. an image acid |
| LSTM-S | nice and love that it's pd hard to find it on amazon now. my face is green and usually i tend |
| Transformer-S | light and smells fantastic! blew up on me a few hours later. just like the hype. it's a watered |
| Own, frozen | so tasty and she's really loved it also it's nice stuff! this day to use it in my ipsy bag that |
| GloVe, frozen | very fizzy and moist. it also tastes too nice and felt as it's a more expensive high end school skincare regimen |
| GloVe, trainable | nice. i will use it really keep it in place. after a few initial loving floral products, i decided |
| Random, frozen | on the flowery so i mixed by the kit within a week. it is advertised to see if this product was |
| Prompt 6: *i bought this for my daughter, who has very curly hair. she said …* | |
| KN 5-gram | she saw my special about it. the part is thick <unk> clear quite difficult to comb. this product was on |
| RNN-S | it is a great deal as well as everyone what i just want to use. i was thinking what in a |
| LSTM-S | that she liked to get the vibrant black black hair clean with the color. it gives you a look of natural |
| Transformer-S | it is a good deal so she loves this product. i don't wear it on a hair tie with no sleep |
| Own, frozen | my hair is very curly at first it's usual dry. i am hard to get it. before using this product |
| GloVe, frozen | that i've noticed her big hair growth in her hair! i've had it and they held well and has no frizz |
| GloVe, trainable | she said this doesn't work, but i am not buying <unk> again. too big for the right length and for |
| Random, frozen | that the hair can't still are coated so i will be a few more to play from everyday, so i decided |

Table D3: The median-of-nine continuation of every generating model for all six prompts, fixed before generation ($T = 1.01$, 22 new tokens). Part II rows: LSTM-S with each input table (seed 4213).

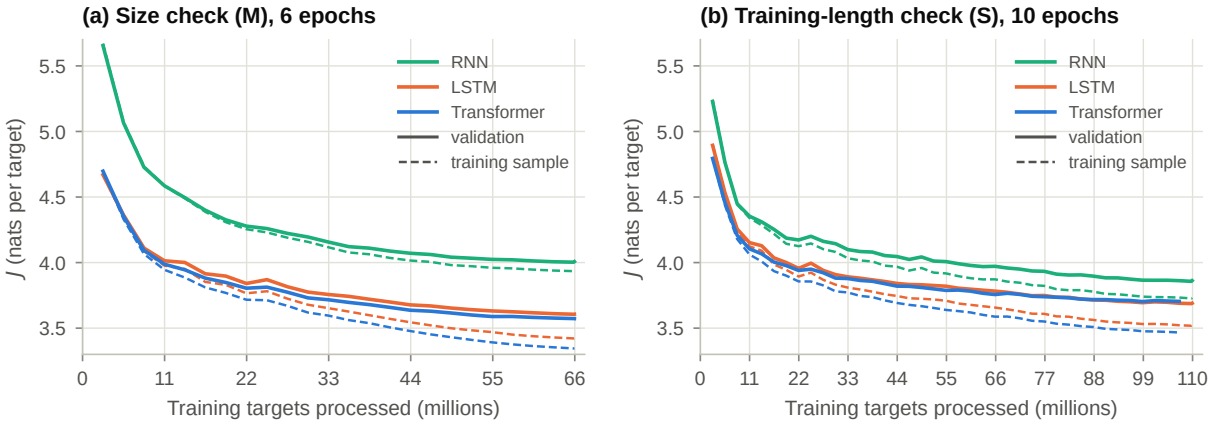



Figure D2: Learning curves of the size check (M) and of the 10-epoch runs at size S; solid validation, dashed the fixed training sample. The 10-epoch Transformer stopped early at 9.7 epochs.

| | Own skip-gram (10.7M training tokens of reviews) | GloVe 2024 (11.9B tokens of Wikipedia and Gigaword) |
|---|---|---|
| WordSim353 Spearman (204 of 353 pairs) | 0.39 | 0.52 |
| Analogies correct (6,863 of 19,544) | 22.1% | 57.5% |
| Neighbours of “cheap” | cheaply, flimsy, cheep, crappy, cheapy | inexpensive, cheaper, expensive, easy, affordable |
| Neighbours of “broke” | snapped, fell, popped, tore, ripped | breaking, break, pulled, came, broken |
| Neighbours of “gift” | present, christmas, stuffer, stocking, birthday | gifts, items, offering, item, item’s |
| Neighbours of “great” | fantastic, good, awesome, wonderful, excellent | greatest, lot’s, good, well, there’s |

Table E1: The two pretrained tables on the course’s word tests, on the 16,544 vocabulary words both have, and their nearest neighbours (cosine).

my vectors and 0.265, 0.256 and 0.250 with GloVe: the order is the same on every seed.

**Stability and tables.** Gradient norms are similar in every setting (mean 0.30 to 0.40); the input table’s own gradient is small (mean norm 0.025 against 0.30 for all parameters). Every fixed table’s fingerprint is identical before and after training. The trainable random table’s root mean square grew from 0.058 to 0.657 (median cosine between a row’s start and end 0.06); trainable GloVe moved less (median cosine 0.55). The `gensim` log reports 309 ignored exceptions: its compiled code mistakes a dot product of exactly $-1.0$ for an error code and uses 0 in that one update, 309 times in roughly ten billion, so I ignore them.

## F Additional Part III Details

**Protocol.** Features are read in evaluation mode with the model’s parameters checked unchanged by fingerprint; feature extraction for the whole test set took 1.0 to 2.8 seconds per model on the GPU. Before the test set was touched, every checkpoint’s validation $J$ was recomputed and matched its record within $10^{-4}$, and an untrained copy built twice gave identical weights. $C$ is searched over $10^{-4}$ to $10^{2}$; if the best value is uniquely at the top it grows tenfold (limit $10^{6}$), it shrinks only when the lowest value strictly beats all others (limit $10^{-8}$), and ties go to the smaller $C$. Folds are stratified by label and grouped by product (seed 4213), identical for every representation, and no product is on both sides of a fold. Word counts keep word and word-pair columns seen in at least two fitting reviews, weighted by tf-idf on each fitting part only. Bootstrap intervals use 2,000 resamples of test reviews shared by every representation; a second set resamples whole products, because reviews of one product are not independent.

**Copies between piles.** Products never cross piles, but short everyday reviews recur across people (kept by design, Appendix B): 6.4% of test texts also appear in the language-model training pile and 5.1% in Part III’s training set, mostly reviews of one or two words (69% of which repeat a Part III training review). A pure lookup of repeated texts would be right on 98.5% of them; the main model scores 98.4% on them against 93.0% on new text. Without reviews under 3 or 5 words, $\tau$ is unchanged for the last word (0.69) and 0.87 and 0.86 for the average.

| Representation | $J$ | Last | Avg. |
|---|---|---|---|
| *Design (S)* | | | |
| RNN-S | 3.914 | 84.9 | 89.3 |
| LSTM-S | 3.708 | 92.3 | 92.2 |
| Transformer-S | 3.718 | 91.5 | 91.8 |
| *Size (M)* | | | |
| RNN-M | 4.001 | 83.8 | 87.9 |
| LSTM-M | 3.602 | 93.4 | 93.0 |
| Transformer-M | 3.570 | 93.3 | 92.9 |
| *Training length (S, 10 epochs)* | | | |
| RNN-S | 3.855 | 87.3 | 90.9 |
| LSTM-S | 3.684 | 93.1 | 92.3 |
| Transformer-S | 3.700 | 91.6 | 91.7 |
| *Word table (LSTM-S)* | | | |
| Own, frozen | 3.811 | 92.0 | 91.6 |
| GloVe, frozen | 3.973 | 89.3 | 89.0 |
| GloVe, trainable | 3.729 | 91.7 | 92.1 |
| Random, frozen | 4.027 | 88.2 | 88.3 |
| *References* | | | |
| Untrained (S) | – | 60.7–66.8 | 71.8–73.3 |
| Tables, averaged | – | | 82.8–89.4 |
| Word counts (153K col.) | – | 93.6 | |
| Length only | – | 52.3 | |

Table F1: Test accuracy (%) at the last word and averaged, seed 4213 (95% intervals $\pm$0.3 to 0.5 points). $J$: language-model test cross-entropy. The seed SD of accuracy, averaged over the three Part II settings, is 0.31 (last word) and 0.23 points (average).

| Pairs in the same order | Last word | Average |
|---|---|---|
| $J$ gap 0 to 0.05 | 9 of 14 | 8 of 14 |
| $J$ gap 0.05 to 0.15 | 19 of 25 | 25 of 25 |
| $J$ gap 0.15 or more | 38 of 39 | 39 of 39 |
| $\tau$ without the RNNs | 0.69 (38 of 45) | 0.78 (40 of 45) |

Table F2: Exploratory checks on the 13-model ranking, not fixed in advance: pairs of models in the same order by test $J$ and by accuracy, by how far apart their $J$ is.

**Top hidden state.** As a check on the readout, I also classified the top hidden state of the six Part I models (100 to 1,120 numbers; Table F4). It gives close results, for example 93.6% for Transformer-M and 93.7% for LSTM-M at the last word, and RNN-S still loses 5.5 points at its last word (85.1% against 90.6% averaged).

**Checks added while writing.** Table F2 was not fixed in advance and is exploratory. With averaged features, every pair of models at least 0.05 nats apart in test $J$ is in the same order. Without the three RNNs, $\tau$ is 0.69 for the last word and 0.78 for the average. Against word counts, uncorrected McNemar tests give $p = 0.29$ (LSTM-M) and 0.07 (Transformer-M) at the last word, and 0.50 and 0.88 for their top hidden states.

**Error categories.** Table F3 gives the counts. The list was fixed before any mistake was read: rating does not match the text; mixed feelings; negation missed; sarcasm; complaint not about the product; too short or vague; other; the first that applies wins. Claude drafted a category and a one-line reason for each of the 40 sampled mistakes; I read every review and decided each category. After that first reading I revised the list, so the final categories are not the ones fixed in advance: "mixed" now requires real good and bad points of the product as it is now (a review that calls one side the only good or bad thing, such as "the only plus" or "only complaint", or praises an older version, belongs to its overall side); the catch-all "other" held reviews any reader can label at once, so it was split by overall side into "negative, but told as plain facts", "negative, but with some praise" and "positive overall"; "negation missed" was dropped (its positive reviews that praise through "not" moved to "positive overall", and its one negative warning to "negative, but told as plain facts"); and "rating does not match" became "stars do not match the text". The 5 plus 5 random disagreements between the main model and word counts follow the same pattern as the examples in Section 4.

| Category (40 random mistakes) | $n$ | 95% interval |
|---|---|---|
| Mixed: real good and bad points | 17 | 29–58% |
| Negative, but told as plain facts | 7 | 9–32% |
| Positive overall | 5 | 5–26% |
| Too short or vague to tell | 5 | 5–26% |
| Negative, but with some praise | 3 | 3–20% |
| Stars do not match the text | 2 | 1–17% |
| Sarcasm | 1 | 0–13% |

Table F3: Categories of 40 random mistakes of Transformer-M at the last word, with 95% Wilson intervals (list fixed in advance, revised after a first reading).

| Representation | Pool | $C$ | CV acc. | Test acc. | 95% interval | Macro-F1 |
|---|---|---|---|---|---|---|
| RNN-S | last word | $10^{-2}$ | 84.9 | 84.9 | 84.5–85.4 | 84.9 |
| RNN-S | average | $10^{1}$ | 89.2 | 89.3 | 88.9–89.7 | 89.3 |
| LSTM-S | last word | $10^{1}$ | 92.1 | 92.3 | 91.9–92.6 | 92.3 |
| LSTM-S | average | $10^{-1}$ | 92.1 | 92.2 | 91.9–92.5 | 92.2 |
| Transformer-S | last word | $10^{2}$ | 91.2 | 91.5 | 91.1–91.8 | 91.5 |
| Transformer-S | average | $10^{1}$ | 91.7 | 91.8 | 91.4–92.1 | 91.8 |
| RNN-M | last word | $10^{-2}$ | 83.5 | 83.8 | 83.4–84.3 | 83.8 |
| RNN-M | average | $10^{2}$ | 88.1 | 87.9 | 87.5–88.3 | 87.9 |
| LSTM-M | last word | $10^{-1}$ | 93.3 | 93.4 | 93.1–93.7 | 93.4 |
| LSTM-M | average | $10^{-1}$ | 92.9 | 93.0 | 92.7–93.4 | 93.0 |
| Transformer-M | last word | $10^{-1}$ | 93.4 | 93.3 | 93.0–93.6 | 93.3 |
| Transformer-M | average | $10^{-1}$ | 92.7 | 92.9 | 92.6–93.2 | 92.9 |
| RNN-S, 10 ep. | last word | $10^{-2}$ | 87.3 | 87.3 | 86.9–87.7 | 87.3 |
| RNN-S, 10 ep. | average | $10^{1}$ | 90.8 | 90.9 | 90.5–91.2 | 90.9 |
| LSTM-S, 10 ep. | last word | $10^{-1}$ | 92.8 | 93.1 | 92.8–93.4 | 93.1 |
| LSTM-S, 10 ep. | average | $10^{-1}$ | 92.2 | 92.3 | 92.0–92.7 | 92.3 |
| Transformer-S, 10 ep. | last word | $10^{-2}$ | 91.5 | 91.6 | 91.3–92.0 | 91.6 |
| Transformer-S, 10 ep. | average | $10^{-1}$ | 91.7 | 91.7 | 91.4–92.1 | 91.7 |
| Own, frozen | last word | $10^{-1}$ | 91.9 | 92.0 | 91.7–92.3 | 92.0 |
| Own, frozen | average | $10^{-2}$ | 91.5 | 91.6 | 91.2–92.0 | 91.6 |
| GloVe, frozen | last word | $10^{-2}$ | 89.1 | 89.3 | 88.9–89.7 | 89.3 |
| GloVe, frozen | average | $10^{1}$ | 88.9 | 89.0 | 88.6–89.4 | 89.0 |
| GloVe, trainable | last word | $10^{2}$ | 91.7 | 91.7 | 91.4–92.1 | 91.7 |
| GloVe, trainable | average | $10^{0}$ | 91.9 | 92.1 | 91.8–92.5 | 92.1 |
| Random, frozen | last word | $10^{-2}$ | 88.3 | 88.2 | 87.8–88.6 | 88.2 |
| Random, frozen | average | $10^{0}$ | 88.4 | 88.3 | 87.9–88.7 | 88.3 |
| Random, trainable (seed 4214) | last word | $10^{-2}$ | 92.6 | 92.8 | 92.4–93.1 | 92.8 |
| Random, trainable (seed 4214) | average | $10^{1}$ | 92.1 | 92.3 | 91.9–92.6 | 92.3 |
| Random, trainable (seed 4215) | last word | $10^{-2}$ | 92.5 | 92.3 | 92.0–92.7 | 92.3 |
| Random, trainable (seed 4215) | average | $10^{-1}$ | 92.3 | 92.4 | 92.0–92.7 | 92.4 |
| Own, frozen (seed 4214) | last word | $10^{1}$ | 91.8 | 91.9 | 91.5–92.2 | 91.9 |
| Own, frozen (seed 4214) | average | $10^{-1}$ | 91.8 | 91.9 | 91.5–92.2 | 91.9 |
| Own, frozen (seed 4215) | last word | $10^{-1}$ | 91.4 | 91.4 | 91.0–91.7 | 91.4 |
| Own, frozen (seed 4215) | average | $10^{-1}$ | 91.8 | 91.8 | 91.4–92.1 | 91.8 |
| GloVe, frozen (seed 4214) | last word | $10^{1}$ | 90.0 | 90.0 | 89.6–90.3 | 90.0 |
| GloVe, frozen (seed 4214) | average | $10^{2}$ | 89.7 | 89.9 | 89.5–90.2 | 89.9 |
| GloVe, frozen (seed 4215) | last word | $10^{-3}$ | 89.6 | 89.6 | 89.2–90.0 | 89.6 |
| GloVe, frozen (seed 4215) | average | $10^{0}$ | 89.6 | 89.6 | 89.3–90.0 | 89.6 |
| Untrained RNN-S | last word | $10^{-1}$ | 60.3 | 60.7 | 60.1–61.3 | 60.6 |
| Untrained RNN-S | average | $10^{1}$ | 72.6 | 72.5 | 71.9–73.1 | 72.5 |
| Untrained LSTM-S | last word | $10^{-1}$ | 66.7 | 66.8 | 66.2–67.4 | 66.8 |
| Untrained LSTM-S | average | $10^{2}$ | 73.0 | 73.3 | 72.8–73.9 | 73.3 |
| Untrained Transformer-S | last word | $10^{2}$ | 62.5 | 62.2 | 61.6–62.8 | 61.9 |
| Untrained Transformer-S | average | $10^{0}$ | 71.8 | 71.8 | 71.2–72.4 | 71.8 |
| Averaged table, LSTM-S's own | | $10^{-1}$ | 87.6 | 87.9 | 87.5–88.3 | 87.9 |
| Averaged table, skip-gram | | $10^{-1}$ | 89.2 | 89.4 | 89.0–89.8 | 89.4 |
| Averaged table, GloVe | | $10^{2}$ | 82.7 | 82.8 | 82.3–83.3 | 82.8 |
| Word counts (tf-idf) | | $10^{1}$ | 93.0 | 93.6 | 93.3–93.9 | 93.6 |
| Length only | | $10^{-4}$ | 53.0 | 52.3 | 51.7–53.0 | 52.3 |
| RNN-S (top hidden) | last word | $10^{-3}$ | 85.3 | 85.1 | 84.7–85.6 | 85.1 |
| RNN-S (top hidden) | average | $10^{1}$ | 90.5 | 90.6 | 90.2–90.9 | 90.6 |
| LSTM-S (top hidden) | last word | $10^{-1}$ | 92.3 | 92.3 | 91.9–92.6 | 92.3 |
| LSTM-S (top hidden) | average | $10^{0}$ | 92.4 | 92.4 | 92.1–92.8 | 92.4 |
| Transformer-S (top hidden) | last word | $10^{-2}$ | 91.2 | 91.4 | 91.1–91.8 | 91.4 |
| Transformer-S (top hidden) | average | $10^{-2}$ | 91.7 | 91.7 | 91.4–92.1 | 91.7 |
| RNN-M (top hidden) | last word | $10^{-3}$ | 84.2 | 84.4 | 83.9–84.8 | 84.4 |
| RNN-M (top hidden) | average | $10^{-1}$ | 90.2 | 90.0 | 89.6–90.3 | 90.0 |
| LSTM-M (top hidden) | last word | $10^{-1}$ | 93.7 | 93.7 | 93.4–94.0 | 93.7 |
| LSTM-M (top hidden) | average | $10^{2}$ | 93.5 | 93.6 | 93.3–93.9 | 93.6 |
| Transformer-M (top hidden) | last word | $10^{-1}$ | 93.6 | 93.6 | 93.3–93.9 | 93.6 |
| Transformer-M (top hidden) | average | $10^{0}$ | 93.1 | 93.2 | 92.9–93.5 | 93.2 |

Table F4: Every representation: chosen $C$, cross-validated accuracy on the training reviews (%), test accuracy (%) with its 95% bootstrap interval, and macro-F1 (%). "untrained": the S model's starting weights; "table": the averaged word table; "top hidden": the appendix check.

| First | Second | Pool | Diff. (pts) | 95% interval | Holm $p$ | Real? |
|---|---|---|---|---|---|---|
| LSTM-S | Untrained LSTM-S | last word | +25.4 | +24.8 to +26.1 | <0.001 | yes |
| LSTM-S | Untrained LSTM-S | average | +18.9 | +18.3 to +19.5 | <0.001 | yes |
| RNN-S | Untrained RNN-S | last word | +24.2 | +23.6 to +24.9 | <0.001 | yes |
| RNN-S | Untrained RNN-S | average | +16.8 | +16.2 to +17.4 | <0.001 | yes |
| Transformer-S | Untrained Transformer-S | last word | +29.3 | +28.6 to +30.0 | <0.001 | yes |
| Transformer-S | Untrained Transformer-S | average | +20.0 | +19.4 to +20.6 | <0.001 | yes |
| RNN-S | LSTM-S | last word | −7.3 | −7.8 to −6.9 | <0.001 | yes |
| RNN-S | LSTM-S | average | −2.9 | −3.2 to −2.6 | <0.001 | yes |
| RNN-S | Transformer-S | last word | −6.6 | −7.0 to −6.1 | <0.001 | yes |
| RNN-S | Transformer-S | average | −2.5 | −2.8 to −2.2 | <0.001 | yes |
| LSTM-S | Transformer-S | last word | +0.8 | +0.4 to +1.1 | <0.001 | yes |
| LSTM-S | Transformer-S | average | +0.4 | +0.1 to +0.7 | 0.078 | no |
| RNN-S | RNN-M | last word | +1.1 | +0.7 to +1.5 | <0.001 | yes |
| RNN-S | RNN-M | average | +1.4 | +1.0 to +1.7 | <0.001 | yes |
| LSTM-S | LSTM-M | last word | −1.2 | −1.4 to −0.9 | <0.001 | yes |
| LSTM-S | LSTM-M | average | −0.8 | −1.1 to −0.6 | <0.001 | yes |
| Transformer-S | Transformer-M | last word | −1.8 | −2.1 to −1.5 | <0.001 | yes |
| Transformer-S | Transformer-M | average | −1.1 | −1.4 to −0.8 | <0.001 | yes |
| RNN-S | RNN-S, 10 ep. | last word | −2.4 | −2.8 to −2.1 | <0.001 | yes |
| RNN-S | RNN-S, 10 ep. | average | −1.6 | −1.9 to −1.3 | <0.001 | yes |
| LSTM-S | LSTM-S, 10 ep. | last word | −0.8 | −1.1 to −0.6 | <0.001 | yes |
| LSTM-S | LSTM-S, 10 ep. | average | −0.1 | −0.4 to +0.1 | 1.000 | no |
| Transformer-S | Transformer-S, 10 ep. | last word | −0.1 | −0.4 to +0.2 | 1.000 | no |
| Transformer-S | Transformer-S, 10 ep. | average | +0.1 | −0.2 to +0.3 | 1.000 | no |
| LSTM-S | Own, frozen | last word | +0.3 | 0.0 to +0.6 | 0.805 | no |
| LSTM-S | Own, frozen | average | +0.6 | +0.3 to +0.9 | <0.001 | yes |
| LSTM-S | GloVe, frozen | last word | +2.9 | +2.6 to +3.3 | <0.001 | yes |
| LSTM-S | GloVe, frozen | average | +3.2 | +2.9 to +3.6 | <0.001 | yes |
| LSTM-S | GloVe, trainable | last word | +0.5 | +0.3 to +0.8 | 0.002 | yes |
| LSTM-S | GloVe, trainable | average | 0.0 | −0.2 to +0.3 | 1.000 | no |
| LSTM-S | Random, frozen | last word | +4.1 | +3.7 to +4.4 | <0.001 | yes |
| LSTM-S | Random, frozen | average | +3.9 | +3.5 to +4.2 | <0.001 | yes |
| RNN-S | Word counts (tf-idf) | last word | −8.7 | −9.2 to −8.2 | <0.001 | yes |
| RNN-S | Word counts (tf-idf) | average | −4.3 | −4.7 to −3.9 | <0.001 | yes |
| LSTM-S | Word counts (tf-idf) | last word | −1.4 | −1.7 to −1.0 | <0.001 | yes |
| LSTM-S | Word counts (tf-idf) | average | −1.4 | −1.8 to −1.1 | <0.001 | yes |
| Transformer-S | Word counts (tf-idf) | last word | −2.1 | −2.5 to −1.8 | <0.001 | yes |
| Transformer-S | Word counts (tf-idf) | average | −1.8 | −2.2 to −1.5 | <0.001 | yes |
| RNN-S | RNN-S | last vs average | −4.4 | −4.8 to −3.9 | <0.001 | yes |
| LSTM-S | LSTM-S | last vs average | +0.1 | −0.3 to +0.4 | 1.000 | no |
| Transformer-S | Transformer-S | last vs average | −0.3 | −0.6 to 0.0 | 0.825 | no |
| RNN-M | RNN-M | last vs average | −4.1 | −4.6 to −3.6 | <0.001 | yes |
| LSTM-M | LSTM-M | last vs average | +0.4 | +0.1 to +0.7 | 0.185 | no |
| Transformer-M | Transformer-M | last vs average | +0.4 | +0.1 to +0.7 | 0.147 | no |
| RNN-S, 10 ep. | RNN-S, 10 ep. | last vs average | −3.5 | −4.0 to −3.1 | <0.001 | yes |
| LSTM-S, 10 ep. | LSTM-S, 10 ep. | last vs average | +0.7 | +0.4 to +1.1 | <0.001 | yes |
| Transformer-S, 10 ep. | Transformer-S, 10 ep. | last vs average | −0.1 | −0.5 to +0.2 | 1.000 | no |
| Own, frozen | Own, frozen | last vs average | +0.4 | +0.1 to +0.7 | 0.211 | no |
| GloVe, frozen | GloVe, frozen | last vs average | +0.3 | 0.0 to +0.7 | 0.753 | no |
| GloVe, trainable | GloVe, trainable | last vs average | −0.4 | −0.8 to −0.1 | 0.211 | no |
| Random, frozen | Random, frozen | last vs average | −0.1 | −0.5 to +0.3 | 1.000 | no |

Table F5: The 51 comparisons fixed in advance: accuracy difference (first minus second, points) with its paired 95% bootstrap interval, exact McNemar $p$ after Holm's correction, and whether the gap is real (significant after Holm and its product-level interval excludes 0).

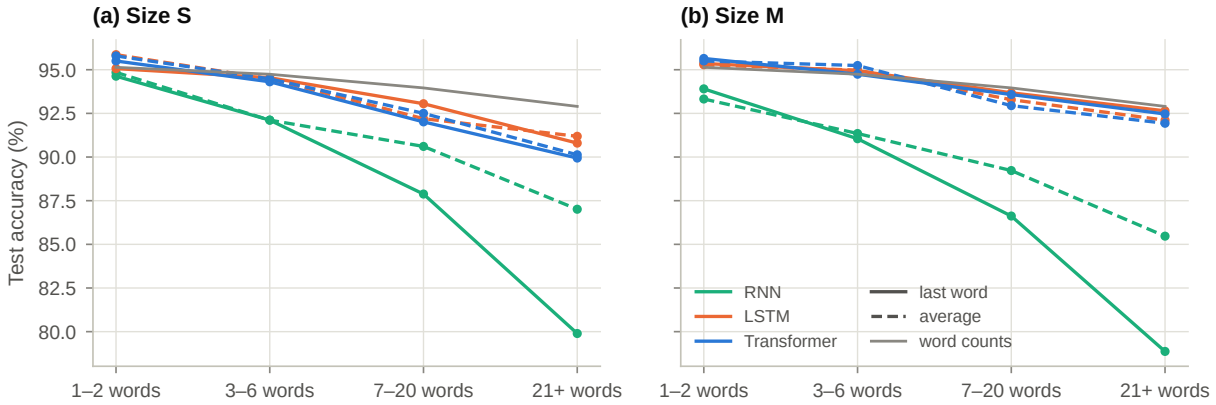



Figure F1: Test accuracy by review length for the S and M models, last word (solid) and average (dashed), with word counts in grey. Only the RNN falls far behind on long reviews when read at its last word.

## G Reproducibility, Data and AI Use

**Code and reproducibility.** The repository (`https://github.com/kr7001/dsa4213-assignment`) holds every script, configuration and result file behind this report; its README lists the data download, every command in order, the configurations and the seeds (4213 throughout, plus 4214 and 4215 for Part II's repeats). It uses Python 3.12, PyTorch 2.14.1 (Paszke et al., 2019), gensim 4.4.0 (Řehůřek and Sojka, 2010) and scikit-learn 1.9.1 (Pedregosa et al., 2011), with every dependency pinned in `uv.lock`. Every run records its git commit, whether the code had uncommitted changes, its configuration, device and machine; the test suite (168 tests) passes. Every table and figure of results is written by `scripts/make_report_figures.py` from the result files, and `scripts/check_report_numbers.py` checks the numbers in this text against them.

**Data.** Amazon Reviews 2023 states no licence, so the repository holds only the download script, never the data, and I cite the dataset (Hou et al., 2024); generated samples and review text stay on my machine, apart from the short quotes and samples shown in this report. GloVe is released under the Open Data Commons Public Domain Dedication and License, and the fastText language-identification model under CC BY-SA 3.0.

**AI use.** I used Claude (Anthropic) throughout, as the brief allows. Claude helped to research different options for the decisions, wrote the code to my design (data pipeline, models, tests and figure scripts), suggested a category for each of the 40 sampled errors, and drafted the report text. I made every design and architecture decision, ran every experiment on my own machine and reviewed its results, decided every error category myself, and revised the report drafts in my own words, style and format. I checked Claude's suggestions against the results and my own reading, and changed them where they did not make sense to me (for example, I revised the error categories).